

FACULTAD DE INGENIERÍA

Escuela Académico Profesional de Ingeniería de Sistemas e Informática

Proyecto de Investigación

**Técnicas de Estructuración de Prompts
en la Calidad de Código Generado por el
Modelo de Lenguaje de Gran Escala
ChatGPT-4o**

Autores:

- Harold Huanca Ccasa
- Jose Leonardo Cruz Gonzales
- Wilder Ronald Ccasani Quispe

Cusco, Perú
2026

INDICE

INDICE DE TABLAS.....	IV
INDICE DE FIGURAS.....	V
INTRODUCCIÓN	VI
CAPITULO I EL PROBLEMA DE INVESTIGACIÓN	1
1.1. Planteamiento del Problema.....	1
1.2. Formulación del Problema	4
1.2.1. Problema General.....	4
1.2.2. Problemas Específicos	4
1.3. Objetivos de la Investigación	4
1.3.1. Objetivo General	4
1.3.2. Objetivos Específicos	5
1.4. Justificación de la Investigación	5
1.4.1. Justificación Teórica.....	5
1.4.2. Justificación Práctica.....	6
1.4.3. Justificación Metodológica	6
1.5. Delimitación de la Investigación.....	7
1.5.1. Delimitación Espacial	7
1.5.2. Delimitación Temporal.....	7
1.5.3. Delimitación Social.....	7
1.6. Hipótesis y Variables.....	8
1.6.1. Hipótesis.....	8
1.6.2. Hipótesis Específicas	8
1.6.3. Definición de Variables	8
1.6.4. Matriz de Consistencia.....	9
1.6.5. Matriz de Operacionalización de Variables	9
CAPITULO II MARCO TEÓRICO	10
2.1. Antecedentes de la Investigación	10
2.1.1. Antecedentes Internacionales.....	10
2.1.2. Antecedentes Nacionales	13
2.2. Bases Teóricas	14

2.2.1.	Inteligencia Artificial Generativa.....	14
2.2.2.	Modelos de Lenguaje de Gran Escala.....	15
2.2.3.	Generación automática de código fuente	17
2.2.4.	Prompt Engineering	18
2.2.5.	Tratamientos experimentales en investigación aplicada	22
2.2.6.	Calidad del software.....	24
2.2.7.	Herramientas de análisis estático	27
2.2.8.	Benchmarks de evaluación.....	30
2.3.	MarcoConceptual.....	33
CAPITULO III METODOLOGÍA		37
3.1.	Enfoque, Tipo, Alcance, Método y Diseño de la Investigación	37
3.1.1.	Enfoque de la Investigación	37
3.1.2.	Tipo de Investigación	37
3.1.3.	Alcance de la Investigación	37
3.1.4.	Método de Investigación	38
3.1.5.	Diseño de la Investigación	38
3.2.	Población y Muestra.....	39
3.2.1.	Población.....	39
3.2.2.	Muestra.....	40
3.3.	Materiales y Métodos.....	40
3.3.1.	Materiales.....	40
3.3.2.	Métodos.....	43
3.4.	Técnicas de Muestreo.....	48
3.5.	Técnicas e Instrumentos de Recolección de Datos	49
3.5.1.	Técnicas de Recolección de Datos.....	49
3.5.2.	Instrumento de Recolección de Datos.....	49
3.6.	Validez y Confiabilidad.....	50
3.6.1.	Validez.....	50
3.6.2.	Confiabilidad.....	51
CAPITULO IV ASPECTOS ADMINISTRATIVOS		53
4.1.	Presupuesto	53
4.2.	Cronograma.....	54

REFERENCIAS.....	56
ANEXOS.....	60
Anexo A. Matriz de Consistencia.....	60
Anexo B. Tabla de Operacionalización de Variables	61
Anexo C. Diagrama de Componentes de la Plataforma de Experimentación..	62
Anexo D. Diagrama de Flujo Experimental(Diagrama de Actividades)	63
Anexo E. Diagrama de Entidad Relación de la Base de Datos experimental	63
Anexo F. Ficha de Registro Experimental	64
Anexo G. Ficha de validación de Experto 1	67
Anexo H. Ficha de validación de Experto 2	68
Anexo I. Ficha de validación de Experto 3	69

INDICE DE TABLAS

TABLA 1.	CALCULO DE LA VALIDEZ POR V DE AIKEN	51
TABLA 2.	TABLA DE PRESUPUESTO DE LA INVESTIGACIÓN.....	53

INDICE DE FIGURAS

FIGURA1. DIAGRAMA DE GANTT DEL CRONOGRAMA DE ACTIVIDADES.....	55
--	-----------

INTRODUCCIÓN

La transformación digital y el avance acelerado de la inteligencia artificial generativa han redefinido la manera en que se desarrollan soluciones tecnológicas en diversos ámbitos del conocimiento. En particular, los modelos de lenguaje de gran escala han demostrado una capacidad creciente para asistir en tareas complejas de programación, permitiendo generar código fuente a partir de instrucciones expresadas en lenguaje natural. Sin embargo, más allá de la funcionalidad inmediata de las soluciones producidas, surge la necesidad de evaluar rigurosamente la calidad del código generado, considerando atributos fundamentales como la exactitud funcional, la mantenibilidad y la complejidad. En este contexto, las técnicas de Prompt Engineering adquieren especial relevancia, debido a que la forma en que se estructuran las instrucciones puede influir significativamente en el comportamiento y desempeño de los modelos de inteligencia artificial. Por ello, resulta necesario desarrollar investigaciones que permitan comprender de manera objetiva el impacto de dichas técnicas sobre la calidad del software generado automáticamente.

El Capítulo I presenta el planteamiento del problema de investigación, abordando la realidad problemática relacionada con el crecimiento del uso de modelos de inteligencia artificial en actividades de programación y las limitaciones existentes respecto a la calidad del código generado. Asimismo, se formulan los problemas de investigación, los objetivos generales y específicos, la justificación del estudio, las hipótesis y la delimitación de la investigación, estableciendo las bases que orientan el desarrollo del trabajo.

El Capítulo II desarrolla el marco teórico que sustenta la investigación. En este apartado se presentan los antecedentes internacionales y nacionales relacionados con la generación automática de código, la inteligencia artificial aplicada al desarrollo de software y las técnicas de Prompt Engineering. Además, se exponen las bases teóricas referentes a los modelos de lenguaje de gran escala, la generación automática de código, la calidad del software y las herramientas utilizadas para su evaluación, complementándose con el marco conceptual de los principales términos empleados en el estudio.

El Capítulo III describe la metodología de investigación adoptada para el desarrollo del estudio. En esta sección se especifican el enfoque, tipo, nivel y diseño de investigación, así como la identificación de las variables, la población y muestra de estudio, las técnicas e instrumentos de recolección de datos y los procedimientos utilizados para la ejecución del experimento y el análisis de los resultados. Asimismo, se detallan las herramientas y métricas empleadas para evaluar la calidad del código generado por el modelo de inteligencia artificial.

El Capítulo IV presenta los aspectos administrativos de la investigación, incluyendo la planificación de actividades, los recursos necesarios para la ejecución del estudio y el presupuesto estimado para su desarrollo. De igual manera, se incorporan las referencias bibliográficas que respaldan científicamente la investigación y los anexos correspondientes, los cuales contienen los instrumentos, matrices y demás documentos complementarios utilizados durante el proceso investigativo.

Finalmente, la investigación busca aportar evidencia empírica sobre la influencia de las técnicas de Prompt Engineering en la calidad del código generado por modelos de inteligencia artificial, contribuyendo al desarrollo de conocimientos aplicables tanto en el ámbito académico como en la práctica profesional de la ingeniería de software.

CAPITULO I

EL PROBLEMA DE INVESTIGACIÓN

1.1. Planteamiento del Problema

En los últimos años, la incorporación de modelos de lenguaje de gran escala (LLMs) dentro del desarrollo de software ha transformado la manera en que los programadores producen código, automatizan tareas y optimizan procesos de ingeniería de software. Las herramientas basadas en inteligencia artificial generativa se utilizan para asistir en la creación de algoritmos, documentación, pruebas unitarias y refactorización de aplicaciones, favoreciendo mejoras en productividad y reducción de tiempos de desarrollo(1). Sin embargo, el crecimiento acelerado de estas tecnologías también ha originado preocupaciones relacionadas con la calidad del código generado, debido a posibles problemas de mantenibilidad, deuda técnica, errores lógicos y vulnerabilidades de seguridad que pueden afectar la confiabilidad del software(la precisión cae de 87.2% en programas cortos de menos de 10 líneas a 26.5% cuando se superan las 50 líneas) (2). Por ello, surge la necesidad de evaluar cómo distintas técnicas de Prompt Engineering influyen en la calidad del código producido.

A nivel internacional, el uso de modelos de lenguaje aplicados al desarrollo de software ha crecido considerablemente por su capacidad para generar código fuente, automatizar procesos de programación y asistir en actividades complejas de ingeniería de software. Al revisar 395 estudios algunos muestran que herramientas basadas en inteligencia artificial generativa pueden producir funciones, pruebas unitarias y componentes completos en diferentes lenguajes, reduciendo tiempos de trabajo y aumentando la productividad(1). No obstante, investigaciones recientes señalan que la calidad del software generado depende en gran medida de la manera en que se estructuran los prompts, ya que técnicas como zero-shot, few-shot o chain-of-thought pueden generar diferencias importantes en mantenibilidad y complejidad(el código generado por IA presenta 34% más complejidad ciclomática), seguridad(22.1% del código generado tiene al menos una vulnerabilidad crítica) y rendimiento del código(el 27.32% del código es más lento en promedio) (3). Frente a ello, investigaciones internacionales buscan desarrollar métodos automatizados de evaluación y buenas prácticas de Prompt Engineering (4).

En el contexto peruano, la adopción de tecnologías de inteligencia artificial y automatización dentro del desarrollo de software viene incrementándose de forma progresiva tanto en instituciones públicas como privadas, impulsada por iniciativas de transformación digital y el crecimiento del sector tecnológico, aunque el 7% de las entidades públicas que mapearon utilizan IA o Ciencia de Datos (5). La utilización de modelos de lenguaje y técnicas de Prompt Engineering ha despertado interés académico y profesional por su potencial para optimizar tareas de programación y reducir tiempos de desarrollo (6). Sin embargo, aún existen limitaciones relacionadas con la evaluación objetiva de la calidad del software generado por inteligencia artificial, especialmente en aspectos vinculados con mantenibilidad, deuda técnica y seguridad del código. Asimismo, todavía son escasas las investigaciones orientadas a comparar técnicas de prompting bajo métricas reconocidas de calidad de software.

A nivel local, la región Cusco viene impulsando progresivamente iniciativas orientadas a la transformación digital y al fortalecimiento de capacidades tecnológicas dentro de instituciones públicas y espacios académicos, promoviendo el uso de herramientas digitales para optimizar procesos y servicios, por ejemplo se destinaron S/ 166,461.81 para el Sistema de Seguridad de la Información (7). Sin embargo, el crecimiento de tecnologías basadas en inteligencia artificial generativa también plantea desafíos relacionados con la calidad y confiabilidad del software desarrollado mediante modelos automatizados. En el ámbito universitario y formativo de la región, el interés por incorporar herramientas de inteligencia artificial en programación y desarrollo de aplicaciones continúa incrementándose; no obstante, todavía existe una limitada investigación orientada a evaluar sistemáticamente cómo las técnicas de Prompt Engineering influyen sobre la calidad del código generado, dificultando el establecimiento de criterios técnicos relacionados con mantenibilidad, seguridad y eficiencia (8).

Actualmente, el uso de modelos de lenguaje de gran escala para la generación automática de código se ha incrementado considerablemente debido a la capacidad de estas tecnologías para asistir en tareas de programación, automatización y desarrollo de software. Sin embargo, aún existe una limitada comprensión sobre el impacto que tienen las distintas técnicas de Prompt Engineering en la calidad del código producido. En muchos casos, los desarrolladores utilizan prompts estructurados de manera empírica,

sin criterios técnicos que permitan garantizar mantenibilidad, seguridad, eficiencia o reducción de deuda técnica (6). Asimismo, investigaciones recientes señalan que el código generado por inteligencia artificial puede presentar problemas de complejidad, errores lógicos, vulnerabilidades y baja mantenibilidad cuando las instrucciones proporcionadas al modelo no son adecuadamente diseñadas, el 37% del código funcional generado en Python presentaba problemas severos de mantenibilidad (2).

Entre las principales causas que originan esta problemática se encuentra el crecimiento acelerado del uso de herramientas de inteligencia artificial generativa dentro del desarrollo de software sin metodologías consolidadas que permitan evaluar objetivamente la calidad del código producido. Asimismo, muchos desarrolladores y organizaciones emplean modelos de lenguaje priorizando resultados funcionales inmediatos, dejando en segundo plano aspectos relacionados con mantenibilidad, seguridad y complejidad del software generado. Otra causa importante corresponde a la limitada disponibilidad de plataformas automatizadas que integren mecanismos de comparación, análisis estático y evaluación cuantitativa de técnicas de Prompt Engineering bajo estándares reconocidos de calidad (3). Del mismo modo, la rápida evolución de los modelos de inteligencia artificial dificulta el establecimiento de criterios homogéneos de validación y gobernanza tecnológica, por ejemplo los prompts basado en instrucciones representan el 37.7% de los problemas de deuda técnica relacionada con prompts (4).

Si esta problemática continúa en el tiempo, el uso creciente de modelos de lenguaje para la generación automática de código podría favorecer el desarrollo de software con mayores niveles de deuda técnica, baja mantenibilidad y vulnerabilidades difíciles de identificar durante etapas tempranas de desarrollo. Asimismo, la ausencia de mecanismos automatizados para evaluar técnicas de Prompt Engineering podría limitar la capacidad de organizaciones y desarrolladores para incorporar prácticas confiables de inteligencia artificial dentro de procesos de ingeniería de software. Esta situación también podría incrementar tiempos asociados a mantenimiento, corrección y refactorización de aplicaciones, todo esto luego de analizar 30 proyectos (9). Del mismo modo, la falta de criterios objetivos para comparar estrategias de prompting dificultaría la toma de decisiones tecnológicas y la integración eficiente de modelos de lenguaje en entornos productivos.

En este contexto, resulta pertinente investigar el impacto de las técnicas de Prompt Engineering sobre la calidad del código generado por modelos de lenguaje de gran escala, debido a la necesidad de establecer mecanismos objetivos que permitan optimizar el uso de inteligencia artificial dentro de la ingeniería de software. La investigación contribuirá a fortalecer el conocimiento relacionado con la evaluación automatizada de software generado por IA, proporcionando evidencia cuantitativa acerca del comportamiento de distintas estrategias de estructuración de prompts bajo métricas reconocidas de calidad. Asimismo, los resultados podrán servir como referencia para desarrolladores, investigadores y organizaciones interesadas en integrar modelos de lenguaje dentro de procesos de desarrollo de software con mayores criterios de mantenibilidad, seguridad y eficiencia.

1.2. Formulación del Problema

1.2.1.Problema General

¿Cómo influyen las técnicas de estructuración de prompts en la calidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o?

1.2.2.Problemas Específicos

PE1. ¿Cómo influyen las técnicas de estructuración de prompts en la exactitud funcional del código generado por el modelo de lenguaje de gran escala ChatGPT-4o?

PE2. ¿Cómo influyen las técnicas de estructuración de prompts en la mantenibilidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o?

PE3. ¿Cómo influyen las técnicas de estructuración de prompts en la complejidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o?

1.3. Objetivos de la Investigación

1.3.1.Objetivo General

Determinar la influencia de las técnicas de estructuración de prompts en la calidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.

1.3.2.Objetivos Específicos

- OE1.** Determinar la influencia de las técnicas de estructuración de prompts en la exactitud funcional del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.
- OE2.** Determinar la influencia de las técnicas de estructuración de prompts en la mantenibilidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.
- OE3.** Determinar la influencia de las técnicas de estructuración de prompts en la complejidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.

1.4. Justificación de la Investigación

1.4.1.Justificación Teórica

La presente investigación posee una justificación teórica debido a que busca fortalecer el conocimiento relacionado con la influencia de las técnicas de estructuración de prompts sobre la calidad del código generado por modelos de lenguaje de gran escala (LLMs), particularmente en tareas vinculadas al desarrollo de software. Estudios sobre arquitecturas basadas en transformers y modelos generativos evidencian que el comportamiento de estos sistemas depende del contexto y de las instrucciones proporcionadas durante la interacción, lo cual influye en los resultados generados (10). Asimismo, investigaciones recientes sobre Prompt Engineering señalan que la manera en que se diseñan las instrucciones puede modificar significativamente el desempeño de los modelos de lenguaje, afectando aspectos relacionados con coherencia, precisión y capacidad de resolución de tareas complejas, incluida la generación automática de código (11).

En ese contexto, aunque diversos trabajos demostraron el potencial de los modelos generativos para asistir actividades de programación, todavía existe limitada evidencia comparativa sobre cómo distintas técnicas de Prompt Engineering, como zero-shot, few-shot, chain-of-thought y role prompting, influyen cuantitativamente en atributos de calidad del software tales como exactitud funcional, mantenibilidad

y complejidad (3)(12) . En consecuencia, la presente investigación contribuirá al fortalecimiento del conocimiento teórico relacionado con inteligencia artificial aplicada a la ingeniería de software, utilizando métricas objetivas y criterios sustentados en estándares de calidad como ISO/IEC 25010 para comprender mejor el impacto de las estrategias de prompting sobre el código generado automáticamente (8).

1.4.2.Justificación Práctica

La investigación presenta una justificación práctica debido a que permitirá desarrollar una plataforma automatizada orientada a la comparación y evaluación de distintas técnicas de estructuración de prompts aplicadas a la generación automática de código mediante modelos de lenguaje. A través de esta plataforma será posible analizar métricas relacionadas con calidad, exactitud funcional, mantenibilidad y complejidad utilizando herramientas automatizadas de análisis estático. Asimismo, los resultados obtenidos podrán servir como referencia para desarrolladores, estudiantes e investigadores interesados en adoptar modelos de inteligencia artificial dentro de procesos de desarrollo de software, facilitando la identificación de estrategias de prompting que generen mejores resultados en términos de calidad y eficiencia (1). De igual manera, la investigación contribuirá al fortalecimiento de buenas prácticas relacionadas con el uso responsable de inteligencia artificial generativa en ingeniería de software, permitiendo optimizar procesos de programación y evaluación automatizada dentro de entornos tecnológicos modernos.

1.4.3.Justificación Metodológica

La presente investigación posee una justificación metodológica debido a que propone un enfoque cuantitativo y experimental orientado a evaluar el impacto de distintas técnicas de estructuración de prompts sobre variables medibles relacionadas con calidad del software generado por inteligencia artificial. El estudio utilizará métricas objetivas obtenidas mediante herramientas de análisis estático y problemas estandarizados provenientes de plataformas reconocidas como LeetCode y HackerRank, permitiendo garantizar condiciones homogéneas de evaluación y comparación experimental. Asimismo, la implementación de una plataforma automatizada facilitará la recolección sistemática de datos, ejecución

de pruebas y generación de resultados comparativos bajo criterios técnicos reproducibles (9). Del mismo modo, la investigación podrá servir como referencia metodológica para futuros estudios relacionados con inteligencia artificial aplicada al desarrollo de software, automatización de procesos y evaluación de calidad en sistemas basados en modelos de lenguaje.

1.5. Delimitación de la Investigación

1.5.1. Delimitación Espacial

La presente investigación se desarrollará en el contexto académico de la sede Cusco de la Universidad Continental, ubicado en el sector de Angostura camino al distrito de Saylla, considerada como entorno institucional de referencia para la ejecución del estudio experimental y el desarrollo de las actividades de investigación vinculadas al área de ingeniería de sistemas e informática. No obstante, debido a la naturaleza tecnológica y experimental del estudio, la delimitación espacial se centrará principalmente en un entorno computacional donde se realizará la generación, análisis y evaluación de fragmentos de código producidos mediante el modelo de lenguaje ChatGPT-4o, empleando plataformas digitales de programación como LeetCode y HackerRank, así como herramientas automatizadas de análisis estático orientadas a medir atributos de calidad del software. En ese sentido, la investigación se circunscribe tanto al espacio académico institucional como al ecosistema digital utilizado para la ejecución del experimento.

1.5.2. Delimitación Temporal

En términos operativos, la investigación comprenderá el periodo entre los meses de julio y noviembre de 2026, abarcando las fases de revisión bibliográfica, diseño metodológico, implementación del entorno experimental, ejecución de pruebas, análisis de resultados y elaboración del informe final de investigación.

1.5.3. Delimitación Social

Para la presente investigación este criterio no aplica, debido a que el estudio se desarrolla bajo un diseño experimental y cuantitativo enfocado en el análisis automatizado de código fuente. Los sujetos de evaluación son fragmentos de

software generados por un modelo de lenguaje y medidos a través de herramientas de análisis estático, por lo que no se involucra a grupos humanos, usuarios o personal de desarrollo dentro de la recolección de datos.

1.6. Hipótesis y Variables

1.6.1.Hipótesis

Las técnicas de estructuración de prompts influyen significativa y positivamente en la calidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.

1.6.2.Hipótesis Específicas

HE1. Las técnicas de estructuración de prompts influyen significativa y positivamente en la exactitud funcional del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.

HE2. Las técnicas de estructuración de prompts influyen significativa y positivamente en la mantenibilidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.

HE3. Las técnicas de estructuración de prompts influyen significativa y positivamente en la complejidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.

1.6.3.Definición de Variables

Variable independiente: Técnicas de estructuración de prompts, conjunto de estrategias utilizadas para formular instrucciones dirigidas a modelos de lenguaje de gran escala con el objetivo de generar código fuente de manera automatizada, optimizando el razonamiento, contextualización y precisión de las respuestas producidas por la inteligencia artificial.

La variable será controlada al aplicar las distintas técnicas de Prompt Engineering sobre problemas estandarizados de programación, determinando la influencia en la variable dependiente posteriormente con la ayuda de los resultados obtenidos en el código generado por modelos de lenguaje.

Variable dependiente: Calidad de código generado, grado en que el código fuente generado por modelos de lenguaje cumple características relacionadas con la exactitud funcional, mantenibilidad y complejidad.

La variable será medida mediante métricas cuantitativas obtenidas a través de herramientas automatizadas de análisis estático aplicadas sobre el código generado por modelos de lenguaje utilizando distintas técnicas de Prompt Engineering.

1.6.4. Matriz de Consistencia

Encontrará el contenido de esta tabla en el Anexo A.

1.6.5. Matriz de Operacionalización de Variables

Usted podrá encontrar el contenido de la operacionalización en el Anexo B.

CAPITULO II

MARCO TEÓRICO

2.1. Antecedentes de la Investigación

2.1.1. Antecedentes Internacionales

Khojah, Neto, Mohamad y Leitner en el año 2025, en el estudio titulado “*The Impact of Prompt Programming on Function-Level Code Generation*” (12), analizaron la influencia de distintas técnicas de Prompt Engineering sobre la calidad del código generado por ChatGPT-4o. El objetivo fue evaluar estrategias como few-shot prompting, chain-of-thought y persona prompting en tareas de programación. La investigación presentó un enfoque cuantitativo con diseño experimental factorial completo, aplicando 32 combinaciones de técnicas sobre 221 tareas del benchmark CoderEval. Los datos fueron recolectados mediante pruebas automatizadas y métricas de correctitud funcional y mantenibilidad. Los resultados evidenciaron que las técnicas basadas en razonamiento paso a paso y contexto adicional mejoraron significativamente la calidad del código generado. Los autores concluyeron que la formulación del prompt influye directamente en el desempeño de los modelos generativos. Este antecedente resulta relevante porque sustenta la relación entre las técnicas de Prompt Engineering y la calidad del código generado (12).

Garcia, Benjamin, Araujo, Santos, Freitas, Almeida, Da Mota, Neto, Li, Chun y Ahmed en el año 2025, en el estudio denominado “*Which Prompting Technique Should I Use? An Empirical Investigation of Prompting Techniques for Software Engineering Tasks*” (13), investigaron el desempeño de diversas técnicas de Prompt Engineering aplicadas a tareas de ingeniería de software. El objetivo fue identificar qué estrategias ofrecían mejores resultados en generación, comprensión y análisis de código. La investigación empleó un enfoque cuantitativo con diseño experimental comparativo, utilizando distintas técnicas de prompting sobre múltiples tareas de desarrollo de software. La evaluación se realizó mediante pruebas automatizadas y métricas de exactitud. Los resultados mostraron que las estrategias few-shot y chain-of-thought mejoraron el rendimiento en tareas complejas de programación. Los autores concluyeron que la efectividad de cada técnica depende del contexto y tipo de problema. Este antecedente respalda la

necesidad de comparar experimentalmente técnicas de prompting y su impacto sobre la calidad del código generado (13).

Paul, Zhu y Bayley en el año 2025, en el estudio titulado *“Investigating The Smells of LLM Generated Code”* (14), analizaron la presencia de code smells y problemas de calidad en código generado por modelos de lenguaje. El objetivo fue identificar defectos recurrentes y evaluar la mantenibilidad del software producido mediante inteligencia artificial generativa. La investigación tuvo un enfoque cuantitativo con diseño experimental, utilizando análisis estático para detectar complejidad, duplicidad y malas prácticas de programación. Los resultados evidenciaron que la calidad del código variaba según la complejidad de la tarea y la forma de estructurar las instrucciones. Los autores concluyeron que los modelos generativos pueden producir soluciones funcionales, aunque mantienen limitaciones relacionadas con calidad interna y sostenibilidad del software. Este antecedente fundamenta la evaluación de métricas asociadas a mantenibilidad y complejidad dentro de la presente investigación (14).

Zhao, Yu y Luo en el año 2025, en el estudio denominado *“A Comparative Empirical Study of Prompting Strategies for Code Generation with Large Language Models”* (15), compararon distintas estrategias de prompting para generación automática de código mediante modelos de lenguaje. El objetivo fue determinar qué técnicas permitían obtener mejores resultados en términos de precisión y calidad del software generado. La investigación utilizó un enfoque cuantitativo con diseño experimental comparativo, evaluando múltiples tareas de programación mediante pruebas funcionales automatizadas. Los resultados mostraron diferencias significativas entre las estrategias evaluadas, destacando que los enfoques basados en razonamiento explícito y ejemplos contextuales generaron soluciones más correctas y estructuradas. Los autores concluyeron que la formulación del prompt constituye un factor determinante en el desempeño de los modelos generativos. Este antecedente respalda la relación causal entre Prompt Engineering y calidad del código generado (15).

Li, Mo, Zeng, Liu, Li y Ma en el año 2026, en el estudio titulado *“A Multi-Dimensional Quality Evaluation of Code Generated by LLM-Based Tools Across Multiple Programming Languages”* (16), evaluaron la calidad del código generado

por herramientas basadas en modelos de lenguaje en diversos lenguajes de programación. El objetivo fue analizar atributos relacionados con mantenibilidad, exactitud funcional y eficiencia. La investigación presentó un enfoque cuantitativo con diseño experimental, empleando herramientas de análisis estático y pruebas funcionales automatizadas. Los resultados evidenciaron variaciones significativas en la calidad del código según el modelo, el lenguaje y el contexto del problema. Los autores concluyeron que la evaluación multidimensional es necesaria para determinar la calidad real del software generado por inteligencia artificial. Este antecedente aporta sustento metodológico al uso de múltiples indicadores para evaluar la calidad del código generado (16).

Coignon, Quinton y Rouvoy en el año 2024, en el estudio titulado “*A Performance Study of LLM-Generated Code on Leetcode*” (17), evaluaron el desempeño del código generado por modelos de lenguaje en problemas algorítmicos de Leetcode. El objetivo fue analizar diferencias en exactitud funcional y eficiencia entre diversos modelos generativos. La investigación tuvo un enfoque cuantitativo con diseño experimental comparativo, utilizando 204 problemas de programación y soluciones producidas por 18 modelos distintos. Los resultados mostraron diferencias moderadas en rendimiento, destacando mejoras en la resolución de problemas complejos mediante técnicas como Chain-of-Thought prompting. Los autores concluyeron que los modelos de lenguaje pueden generar soluciones competitivas, aunque persisten limitaciones relacionadas con calidad y consistencia. Este antecedente respalda el uso de plataformas como Leetcode para la evaluación experimental del código generado (17).

Asfandiyar, Akhtar y Khalil (3), en su investigación de 2025 titulada *Evaluating the Impact of AI Code Generated of Software Quality and Observed Study of Maintainability, Security and Performance*, evaluaron la calidad del código generado por modelos como GPT-4 y Claude 3. Mediante un enfoque experimental cuantitativo compararon 642 soluciones generadas por IA con 107 desarrolladas por humanos utilizando herramientas como SonarQube y Bandit. Los resultados mostraron un incremento del 34 % en complejidad ciclomática y una densidad de code smells superior al doble respecto al código humano. Asimismo, identificaron una correlación positiva entre complejidad de la tarea y degradación de la calidad.

Los autores concluyeron que el uso de IA puede generar deuda técnica significativa si no se aplican mecanismos de control adecuados. Este antecedente valida el uso de métricas relacionadas con mantenibilidad y complejidad dentro de la presente investigación...

Aljohani y Do (4), en su investigación de 2025 titulada PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects, analizaron la deuda técnica asociada al uso de modelos de lenguaje en proyectos de software. Mediante el estudio de 93,142 archivos Python identificaron que el 54.49 % de las instancias de deuda técnica provenían de integraciones con OpenAI y el 12.35 % de LangChain. Los resultados evidenciaron que el diseño de prompts constituye una fuente importante de deuda técnica, especialmente en técnicas basadas en instrucciones y few-shot prompting. Los autores concluyeron que la acumulación de configuraciones temporales y ajustes no estructurados incrementa la carga de mantenimiento del software. Este antecedente fundamenta la importancia de estudiar cómo distintas técnicas de Prompt Engineering pueden influir en la calidad del código generado

Azeem, Naveed, Sajid y Ali (18), en su estudio de 2025 titulado AI vs. Human Programmers: Complexity and Performance in Code Generation, compararon código generado por GPT-4 con soluciones desarrolladas por programadores expertos. La investigación analizó 300 programas en Python utilizando métricas como Complejidad Ciclomática y Complejidad de Halstead. Los resultados mostraron que el código generado por IA era más extenso y complejo, registrando incrementos del 20.8 % en operadores y 29.2 % en operandos respecto al código humano. Aunque la IA obtuvo mejores niveles de legibilidad, los desarrolladores superaron al modelo en aspectos relacionados con estructura lógica y manejo de errores. Los autores concluyeron que el código generado por IA requiere validación y refinamiento para cumplir estándares de calidad. Este antecedente respalda el uso de métricas de complejidad y la necesidad de procedimientos sistemáticos de generación de código...

2.1.2. Antecedentes Nacionales

Gutiérrez Chávez [7], en su investigación de 2024 titulada Análisis de la aplicación de prácticas ágiles y la gestión de la deuda técnica en el desarrollo de software en MediaLab, analizó la relación entre prácticas ágiles y deuda técnica en proyectos de software. La investigación presentó un enfoque cuantitativo con diseño no experimental, empleando encuestas y análisis estático mediante SonarQube. Los resultados evidenciaron que la ausencia de estrategias formales de gestión de deuda técnica incrementa los costos de mantenimiento y afecta la sostenibilidad del software. Asimismo, se identificó que la presión por cumplir plazos favorece la acumulación de deuda técnica. Se concluyó que es necesario implementar mecanismos preventivos y herramientas de análisis continuo para mejorar la calidad del software. Este antecedente respalda el uso de análisis estático para evaluar mantenibilidad y deuda técnica en la presente investigación...

Pardo Mesías (19), en su estudio de 2018 titulado Mantenibilidad de productos de software según el modelo SQUARE ISO/IEC 25000 evaluó la mantenibilidad de productos de software desarrollados por egresados de ingeniería informática en el Perú. La investigación fue de tipo aplicada con nivel descriptivo y utilizó herramientas de análisis estático basadas en métricas de la norma ISO/IEC 25000. Los resultados mostraron niveles adecuados de mantenibilidad, aunque se identificaron deficiencias en modularidad y capacidad de prueba. El autor concluyó que la aplicación de modelos normativos permite evaluar de forma estructurada la calidad del software y orientar procesos de mejora continua. Este antecedente sustenta la evaluación de la mantenibilidad como componente fundamental de la calidad del código generado...

2.2. Bases Teóricas

2.2.1. Inteligencia Artificial Generativa

2.2.1.1. Concepto de Inteligencia Artificial Generativa

La inteligencia artificial generativa constituye una rama avanzada de la inteligencia artificial orientada a la creación automática de contenido a partir de patrones aprendidos mediante grandes volúmenes de datos. A diferencia de los sistemas tradicionales basados principalmente en clasificación, reconocimiento o predicción, los modelos generativos

poseen la capacidad de producir contenido original como texto, imágenes, audio y código fuente mediante procesos de aprendizaje profundo. En los últimos años, esta tecnología ha experimentado un crecimiento significativo debido al desarrollo de arquitecturas basadas en Transformers y modelos fundacionales capaces de comprender lenguaje natural, contexto y relaciones semánticas complejas, ampliando sus posibilidades de aplicación en distintos campos tecnológicos (12).

2.2.1.2. Aplicaciones de Ingeniería de Software

Dentro del ámbito de la ingeniería de software, la inteligencia artificial generativa comenzó a desempeñar un rol importante en actividades relacionadas con asistencia al programador, generación automática de código, documentación, depuración de software y automatización de tareas repetitivas, contribuyendo a reducir tiempos de desarrollo y mejorar la productividad en distintos entornos de programación. Sin embargo, diversos estudios evidenciaron que el contenido generado por estos modelos puede presentar limitaciones relacionadas con exactitud funcional, mantenibilidad, vulnerabilidades de seguridad y deuda técnica, especialmente cuando las instrucciones proporcionadas al sistema no se encuentran adecuadamente estructuradas (14). Debido a ello, la investigación reciente no solo se orienta hacia las capacidades generativas de estos modelos, sino también hacia la evaluación de la calidad del software producido y de las técnicas utilizadas para optimizar sus resultados.

2.2.2. Modelos de Lenguaje de Gran Escala

2.2.2.1. Definición de Modelos de Lenguaje de Gran Escala

Los modelos de lenguaje de gran escala, conocidos como *Large Language Models* (LLMs), representan sistemas de inteligencia artificial entrenados mediante enormes volúmenes de información textual con el propósito de comprender, predecir y generar lenguaje natural de forma coherente. Estos modelos poseen capacidades para interpretar contexto, identificar

relaciones semánticas complejas y producir respuestas consistentes a partir de instrucciones expresadas en lenguaje natural, ampliando significativamente las posibilidades de automatización en distintos ámbitos tecnológicos (13).

2.2.2.2. Funcionamiento basado en arquitecturas Transformer

Los LLMs utilizan arquitecturas basadas en Transformer, las cuales permiten identificar relaciones contextuales complejas dentro de secuencias de texto mediante mecanismos de atención que favorecen la comprensión del significado y contexto de las instrucciones proporcionadas. Gracias a esta arquitectura, los modelos son capaces de ejecutar tareas relacionadas con generación de contenido, traducción automática, razonamiento, asistencia conversacional y programación automatizada, demostrando una alta capacidad de adaptación frente a distintos tipos de problemas computacionales (13).

2.2.2.3. Aplicación de los LLMs en el desarrollo de software

En el campo del desarrollo de software, los LLMs comenzaron a emplearse para la generación automática de código fuente, proporcionando soluciones capaces de interpretar instrucciones escritas en lenguaje natural y transformarlas en programas funcionales. Herramientas basadas en modelos como GPT, CodeLlama o StarCoder evidenciaron un desempeño competitivo en tareas relacionadas con programación, resolución de problemas algorítmicos y generación de funciones completas, favoreciendo procesos de automatización y asistencia al programador (17).

2.2.2.4. Relación entre LLMs y Prompt Engineering

Diversos estudios señalaron que el rendimiento de los modelos de lenguaje depende considerablemente de la forma en que se estructuran las instrucciones o *prompts* proporcionados al sistema, debido a que pequeñas variaciones en el contexto pueden generar diferencias importantes en la calidad, coherencia y exactitud de las respuestas producidas (15). En consecuencia, los LLMs no solo son considerados herramientas de

automatización, sino también componentes centrales dentro de investigaciones relacionadas con *Prompt Engineering*, generación automática de código y evaluación de calidad del software generado mediante inteligencia artificial.

2.2.3. Generación automática de código fuente

2.2.3.1. Definición de generación automática de código fuente

La generación automática de código fuente mediante inteligencia artificial consiste en el uso de modelos computacionales capaces de producir fragmentos de software a partir de instrucciones expresadas en lenguaje natural o especificaciones previamente definidas. Esta capacidad surgió como resultado de la evolución de los modelos de lenguaje de gran escala y del incremento en la disponibilidad de repositorios públicos de código utilizados durante los procesos de entrenamiento, permitiendo a los sistemas aprender patrones, estructuras sintácticas y soluciones empleadas dentro del desarrollo de software (16)

2.2.3.2. Aplicaciones de la generación automática de código

En la actualidad, los sistemas de generación automática son capaces de resolver problemas algorítmicos, crear funciones completas y asistir en actividades relacionadas con documentación, depuración y optimización de software. Diversas investigaciones demostraron que estas herramientas pueden incrementar la productividad de los desarrolladores, facilitar la automatización de tareas repetitivas y reducir el tiempo requerido para implementar soluciones computacionales, favoreciendo procesos de desarrollo más ágiles y eficientes (17).

2.2.3.3. Limitaciones relacionadas con la calidad del código generado

A pesar de sus ventajas, diversos estudios identificaron limitaciones relacionadas con la calidad estructural del código generado mediante inteligencia artificial, especialmente en aspectos vinculados con mantenibilidad, seguridad, complejidad y exactitud funcional. Debido a ello, múltiples investigaciones comenzaron a evaluar el software

producido mediante modelos generativos utilizando métricas automatizadas y herramientas de análisis estático, con el propósito de medir objetivamente la calidad del código y reducir riesgos asociados a vulnerabilidades, deuda técnica o dificultades de mantenimiento (14).

2.2.3.4. Relación entre generación automática de código y Prompt Engineering

Investigaciones recientes evidenciaron que la forma en que se diseñan las instrucciones dirigidas al modelo influye significativamente en el desempeño y calidad del código generado, debido a que pequeñas variaciones en el contexto pueden producir diferencias importantes en exactitud, estructura y comportamiento funcional de las soluciones propuestas (12). En consecuencia, las técnicas de Prompt Engineering adquirieron relevancia dentro del campo de la ingeniería de software y la investigación aplicada a modelos generativos, al permitir optimizar la interacción con los modelos de lenguaje para obtener resultados más adecuados y consistentes.

2.2.4. Prompt Engineering

El Prompt Engineering, también denominado ingeniería de prompts, corresponde al conjunto de técnicas utilizadas para diseñar y estructurar instrucciones dirigidas a modelos de lenguaje de gran escala con la finalidad de optimizar la calidad, coherencia y precisión de las respuestas generadas por sistemas de inteligencia artificial (13). En el contexto de la generación automática de código fuente, esta disciplina adquirió gran relevancia debido a que el desempeño de los modelos depende considerablemente de la manera en que se formula el prompt entregado al sistema. Diversas investigaciones evidenciaron que pequeñas variaciones en la estructura de las instrucciones pueden modificar significativamente la exactitud funcional, mantenibilidad y eficiencia del código producido por los modelos generativos (12). Debido a ello, el Prompt Engineering comenzó a ser considerado un componente estratégico dentro de la ingeniería de software asistida por inteligencia artificial, permitiendo mejorar procesos relacionados con razonamiento lógico, resolución de problemas y generación automatizada de soluciones computacionales.

En la actualidad, las técnicas de Prompt Engineering son utilizadas en tareas relacionadas con síntesis de código, documentación automática, depuración de errores y análisis de software, buscando reducir ambigüedades y proporcionar contexto suficiente al modelo generativo (15). Asimismo, estudios recientes demostraron que ciertas estrategias de prompting permiten obtener resultados considerablemente superiores respecto a prompts simples o ambiguos, especialmente en problemas de programación complejos que requieren razonamiento secuencial o interpretación contextual (13). En consecuencia, la ingeniería de prompts se consolidó como una línea de investigación relevante dentro de la inteligencia artificial aplicada al desarrollo de software, debido a su capacidad para influir directamente sobre la calidad del código generado automáticamente.

2.2.4.1. Zero-shot

El zero-shot prompting corresponde a una técnica de estructuración de prompts en la cual el modelo de lenguaje recibe únicamente una instrucción o descripción del problema sin incluir ejemplos previos de resolución. Este enfoque busca aprovechar las capacidades generales de razonamiento y aprendizaje implícito adquiridas durante el entrenamiento del modelo, permitiendo generar respuestas directamente a partir del contexto textual proporcionado (15). Dentro del ámbito de generación automática de código, esta técnica es utilizada frecuentemente para resolver problemas simples o tareas donde el modelo posee suficiente conocimiento previo sobre patrones comunes de programación.

Diversas investigaciones señalaron que el zero-shot prompting presenta ventajas relacionadas con simplicidad, rapidez de implementación y menor consumo de tokens; sin embargo, también evidenciaron limitaciones importantes en tareas complejas que requieren razonamiento estructurado o comprensión detallada del contexto (12). En estudios comparativos relacionados con generación de código, se observó que esta técnica puede producir soluciones funcionales, aunque con niveles variables de exactitud y calidad dependiendo de la complejidad del problema solicitado (17). Debido a ello, el zero-shot prompting suele

utilizarse como línea base experimental para comparar el rendimiento de otras estrategias más avanzadas de Prompt Engineering.

2.2.4.2. Few-shot

El few-shot prompting consiste en proporcionar al modelo de lenguaje uno o varios ejemplos previos antes de plantear la tarea principal que se desea resolver. Mediante esta técnica, el sistema puede identificar patrones estructurales y contextuales presentes en los ejemplos suministrados, mejorando su capacidad para generar respuestas coherentes y alineadas con el comportamiento esperado(15). En el ámbito de la generación automática de código, el few-shot prompting se utiliza para orientar al modelo hacia determinados estilos de programación, estructuras lógicas o formatos específicos de solución.

Estudios recientes demostraron que la inclusión de ejemplos dentro del prompt puede incrementar significativamente la exactitud funcional y la calidad estructural del código generado, especialmente en tareas complejas relacionadas con algoritmos y resolución de problemas computacionales (12). Asimismo, investigaciones comparativas evidenciaron que esta técnica tiende a producir soluciones más consistentes respecto al zero-shot prompting, debido a que reduce ambigüedades y proporciona contexto adicional al modelo generativo (15). No obstante, también se identificó que el desempeño del few-shot prompting depende de factores como cantidad de ejemplos utilizados, calidad contextual y similitud entre las tareas presentadas y el problema objetivo.

2.2.4.3. Chain-of-thought

El Chain-of-Thought prompting es una técnica de Prompt Engineering basada en inducir al modelo de lenguaje a desarrollar un razonamiento secuencial y explícito antes de generar una respuesta final. Para ello, el prompt incorpora instrucciones orientadas a que el sistema explique paso a paso el proceso lógico utilizado para resolver un problema determinado (13). Esta estrategia adquirió gran relevancia en tareas relacionadas con

razonamiento matemático, análisis lógico y generación automática de código, debido a que permite mejorar la comprensión contextual y reducir errores derivados de respuestas generadas de forma inmediata.

Diversos estudios experimentales evidenciaron que el Chain-of-Thought prompting incrementa la capacidad de los modelos para resolver problemas complejos y generar soluciones más correctas y estructuradas (12). En investigaciones relacionadas con programación automática, esta técnica mostró mejoras importantes en exactitud funcional y coherencia lógica del código producido, especialmente en problemas algorítmicos de múltiples etapas (17). Asimismo, investigaciones comparativas concluyeron que el razonamiento explícito favorece la generación de soluciones más comprensibles y mantenibles, aunque también puede incrementar el tiempo de procesamiento y la longitud de las respuestas generadas (15).

2.2.4.4. Role prompting

El role prompting consiste en asignar al modelo de lenguaje un rol específico antes de solicitar la resolución de una tarea determinada. Mediante esta técnica, el sistema adopta un comportamiento contextual relacionado con el rol indicado, lo que permite orientar las respuestas hacia determinados estilos, niveles de especialización o enfoques de razonamiento (12). En tareas de generación automática de código, es común utilizar instrucciones como “actúa como un ingeniero de software senior” o “compórtate como un experto en seguridad informática” con el propósito de influir sobre la calidad y estructura del software generado.

Investigaciones recientes demostraron que el role prompting puede mejorar la coherencia contextual y favorecer la producción de código más alineado con buenas prácticas de desarrollo (13). Asimismo, estudios experimentales identificaron que la asignación de roles especializados contribuye a reducir errores lógicos y mejorar aspectos relacionados con mantenibilidad y claridad estructural del código (15). Sin embargo, también se observó que el impacto de esta técnica depende

considerablemente de la precisión del rol definido y del contexto proporcionado al modelo. Debido a ello, el role prompting es considerado actualmente una estrategia relevante dentro de la ingeniería de prompts aplicada a la generación automática de software.

2.2.4.5. Técnicas de estructuración de prompts como estrategia de optimización

Las técnicas de estructuración de prompts representan estrategias orientadas a organizar las instrucciones proporcionadas a los modelos de lenguaje de gran escala con el propósito de mejorar la precisión, coherencia y calidad de las respuestas generadas. En el contexto de generación automática de código, estas técnicas permiten reducir problemas relacionados con instrucciones ambiguas, errores funcionales y baja calidad del software producido. Según Khojah et al.(12), la construcción estructurada de prompts, denominada prompt programming, influye en características como corrección funcional, similitud y calidad del código generado.

Dentro de la generación automática de código, las técnicas de prompting pueden ser consideradas tratamientos experimentales debido a que modifican la interacción con el modelo manteniendo condiciones controladas de evaluación. Khojah et al. (12) analizaron técnicas como few-shot learning, chain-of-thought y persona prompting, evidenciando que su aplicación puede generar diferencias en la corrección y calidad del código producido. Sin embargo, los autores señalan que combinar múltiples técnicas no siempre mejora los resultados, debido a que cada estrategia puede influir de manera diferente sobre atributos como mantenibilidad y complejidad.

2.2.5.Tratamientos experimentales en investigación aplicada

2.2.5.1. Concepto de Tratamiento experimental

Los tratamientos experimentales corresponden a las condiciones o estímulos que el investigador aplica deliberadamente sobre una unidad experimental con la finalidad de analizar sus efectos sobre una variable

dependiente. Dentro de los diseños experimentales, el investigador manipula una variable independiente y observa los cambios producidos en la variable dependiente bajo condiciones controladas, permitiendo establecer relaciones de influencia entre los elementos estudiados (20).

En este sentido, un tratamiento representa una modalidad específica de aplicación de la variable independiente dentro de un experimento. La comparación entre diferentes tratamientos permite identificar si existen diferencias significativas en los resultados obtenidos, manteniendo constantes otros factores que podrían afectar el comportamiento de la unidad experimental (20).

2.2.5.2. Aplicación de tratamientos experimentales en estudios de generación automática de código

En investigaciones relacionadas con modelos de lenguaje de gran escala, los tratamientos experimentales pueden representarse mediante diferentes estrategias de interacción aplicadas sobre el modelo generativo. Estas condiciones permiten evaluar cómo las variaciones en la construcción de instrucciones influyen sobre características del código generado, como exactitud funcional, calidad y mantenibilidad. Khojah et al. (2025) aplicaron un diseño experimental donde diferentes técnicas de prompting fueron utilizadas como tratamientos para analizar su impacto sobre código generado por modelos como GPT-4o, Llama3 y Mistral (12).

Asimismo, los autores evaluaron combinaciones de técnicas como *few-shot learning*, *chain-of-thought* y *persona prompting*, considerando cada configuración del prompt como una condición experimental diferente. Los resultados evidenciaron que las estrategias de estructuración del prompt pueden modificar ciertos atributos del código generado, aunque su efecto depende del criterio de evaluación utilizado, debido a que una técnica puede favorecer la corrección funcional mientras otra puede influir en aspectos relacionados con la calidad del software (12).

En la presente investigación, las técnicas de estructuración de prompts serán consideradas tratamientos experimentales aplicados al modelo de

lenguaje ChatGPT-4o, donde cada estrategia representa una condición controlada para evaluar su influencia sobre la calidad del código generado. De esta manera, los tratamientos permiten comparar el comportamiento del modelo bajo diferentes formas de interacción y determinar cuáles generan mejores resultados según las dimensiones definidas: exactitud funcional, mantenibilidad y complejidad.

2.2.6. Calidad del software

La calidad del software corresponde al conjunto de características y atributos que determinan el grado en que un producto de software satisface requisitos funcionales y no funcionales establecidos para cumplir adecuadamente las necesidades de los usuarios y del entorno donde será utilizado. Dentro de la ingeniería de software, la calidad constituye uno de los aspectos más relevantes debido a que influye directamente sobre la eficiencia, confiabilidad, mantenibilidad, seguridad y rendimiento de los sistemas informáticos (16). En la actualidad, la evaluación de calidad adquirió mayor importancia debido al incremento en el uso de herramientas de generación automática de código basadas en inteligencia artificial, las cuales son capaces de producir soluciones funcionales, aunque no necesariamente sostenibles o seguras desde una perspectiva de ingeniería de software (14).

Diversas investigaciones señalaron que el código generado mediante modelos de lenguaje puede presentar problemas relacionados con complejidad excesiva, vulnerabilidades, duplicidad de código y deuda técnica, especialmente cuando no existen mecanismos adecuados de validación y análisis automatizado (12). Debido a ello, comenzaron a utilizarse modelos y estándares internacionales orientados a evaluar atributos de calidad del software de manera objetiva. Entre dichos modelos, la norma ISO/IEC 25010 se consolidó como uno de los principales referentes para medir características relacionadas con mantenibilidad, confiabilidad, seguridad y calidad interna del software. Asimismo, múltiples estudios recientes utilizaron herramientas de análisis estático y pruebas automatizadas con el propósito de medir la calidad del código generado por modelos de inteligencia artificial (16). En consecuencia, la calidad del software se convirtió en un componente fundamental dentro de las investigaciones relacionadas con generación automática de código y

Prompt Engineering, debido a que permite evaluar de forma cuantificable el impacto de distintas técnicas de interacción con modelos generativos.

2.2.6.1. Exactitud Funcional

La exactitud funcional corresponde al grado en que un sistema de software es capaz de cumplir correctamente las funciones y requisitos para los cuales fue diseñado. Dentro de las investigaciones relacionadas con generación automática de código, esta dimensión es utilizada para determinar si las soluciones generadas por modelos de lenguaje resuelven adecuadamente el problema planteado y producen resultados correctos durante su ejecución (17). La exactitud funcional suele medirse mediante pruebas automatizadas, casos de prueba y validaciones comparativas frente a soluciones esperadas.

Diversos estudios experimentales evidenciaron que la exactitud funcional del código generado por inteligencia artificial puede variar considerablemente dependiendo del modelo utilizado y de las técnicas de prompting aplicadas (12). Asimismo, investigaciones relacionadas con Chain-of-Thought prompting demostraron que el razonamiento paso a paso puede mejorar significativamente la capacidad de los modelos para resolver problemas complejos de programación (13). Debido a ello, la exactitud funcional constituye una dimensión crítica dentro de investigaciones orientadas a comparar el impacto de distintas estrategias de Prompt Engineering sobre la calidad del código generado automáticamente.

2.2.6.2. Mantenibilidad

La mantenibilidad es una característica de calidad del software relacionada con la facilidad con la que un sistema puede ser comprendido, corregido, modificado o ampliado durante su ciclo de vida. De acuerdo con los modelos de calidad de software, esta dimensión resulta esencial para garantizar sostenibilidad y evolución continua de los sistemas informáticos (16). En el contexto de generación automática de código

mediante inteligencia artificial, la mantenibilidad adquirió relevancia debido a que los modelos generativos pueden producir soluciones funcionales, aunque estructuralmente difíciles de interpretar o modificar posteriormente.

Diversos estudios evidenciaron que ciertos fragmentos de código generados por modelos de lenguaje presentan problemas relacionados con redundancia, complejidad excesiva y baja legibilidad, afectando negativamente los procesos de mantenimiento y evolución del software (14). Por esta razón, investigaciones recientes comenzaron a utilizar métricas automatizadas como Maintainability Index y herramientas de análisis estático para evaluar la calidad estructural del código generado automáticamente (12). En consecuencia, la mantenibilidad constituye una dimensión relevante para determinar si las técnicas de Prompt Engineering influyen sobre la producción de código más comprensible y sostenible desde una perspectiva de ingeniería de software.

2.2.6.3. Complejidad

La complejidad del software corresponde al grado de dificultad asociado a la comprensión, desarrollo, prueba, mantenimiento y evolución de un sistema informático. Esta característica está relacionada con la estructura lógica del código, la cantidad de interacciones entre sus componentes y el nivel de esfuerzo requerido para analizar su comportamiento. Dentro de la ingeniería de software, la complejidad constituye un atributo relevante debido a que influye directamente sobre la calidad, mantenibilidad y confiabilidad de las aplicaciones, ya que sistemas excesivamente complejos tienden a presentar mayores probabilidades de errores, dificultades de modificación y costos elevados de mantenimiento (16).

En el contexto de la generación automática de código mediante modelos de lenguaje de gran escala, la complejidad adquiere especial importancia debido a que las soluciones generadas pueden resolver correctamente un problema funcional, pero presentar estructuras lógicas innecesariamente extensas o difíciles de comprender. Diversas investigaciones evidenciaron

que algunos modelos generativos producen código con mayores niveles de complejidad respecto a soluciones desarrolladas por programadores humanos, incrementando el esfuerzo requerido para su validación, prueba y mantenimiento posterior (3). Asimismo, estudios recientes señalaron que la estructura de las instrucciones proporcionadas al modelo puede influir significativamente sobre la complejidad del código generado, favoreciendo la producción de soluciones más organizadas y comprensibles cuando se emplean técnicas avanzadas de Prompt Engineering (15).

La evaluación de la complejidad del software suele realizarse mediante métricas objetivas obtenidas a través de herramientas de análisis estático. Entre las métricas más utilizadas se encuentra la complejidad ciclomática propuesta por McCabe, la cual permite cuantificar la cantidad de caminos lógicos independientes presentes dentro de un programa y estimar el esfuerzo requerido para comprender, probar y mantener el código fuente (14). Debido a ello, la complejidad constituye una dimensión relevante para evaluar la calidad del software generado automáticamente y determinar la influencia que ejercen las distintas técnicas de estructuración de prompts sobre la producción de soluciones computacionales más simples, comprensibles y sostenibles desde una perspectiva de ingeniería de software (12).

2.2.7.Herramientas de análisis estático

Las herramientas de análisis estático corresponden a sistemas especializados en evaluar código fuente sin necesidad de ejecutar el programa, permitiendo identificar problemas relacionados con calidad, seguridad, mantenibilidad y complejidad estructural del software. Dentro de la ingeniería de software, estas herramientas adquirieron gran relevancia debido a su capacidad para detectar vulnerabilidades, malas prácticas de programación y deficiencias técnicas de manera automatizada (14). En investigaciones relacionadas con generación automática de código mediante inteligencia artificial, el análisis estático comenzó a utilizarse como mecanismo objetivo para medir la calidad del software producido por el modelo de lenguaje de gran escala ChatGPT-4o.

Diversos estudios recientes emplearon herramientas automatizadas para evaluar atributos de calidad en código generado por inteligencia artificial, permitiendo cuantificar métricas relacionadas con mantenibilidad, complejidad ciclomática, deuda técnica y vulnerabilidades de seguridad (16). Asimismo, investigaciones experimentales demostraron que el uso de herramientas de evaluación automática facilita la comparación objetiva entre distintas técnicas de Prompt Engineering y diferentes modelos generativos (12). Debido a ello, el análisis estático se consolidó como un componente fundamental dentro de estudios orientados a evaluar calidad de software generado automáticamente.

2.2.7.1. Pruebas Unitarias Automatizadas

Las pruebas unitarias automatizadas corresponden a procedimientos utilizados para verificar el correcto funcionamiento de componentes específicos dentro de un sistema software mediante casos de prueba ejecutados automáticamente. Estas pruebas permiten validar si las funciones y algoritmos implementados cumplen adecuadamente los requisitos esperados, constituyendo uno de los mecanismos más utilizados para evaluar exactitud funcional del software (17). En investigaciones relacionadas con generación automática de código mediante inteligencia artificial, las pruebas unitarias adquirieron gran importancia debido a que permiten determinar objetivamente si las soluciones producidas por modelos de lenguaje resuelven correctamente los problemas planteados.

Diversos estudios experimentales utilizaron conjuntos de test cases automatizados para comparar el rendimiento funcional de distintas técnicas de prompting y modelos generativos (12). Asimismo, investigaciones relacionadas con generación de código algorítmico demostraron que el porcentaje de pruebas superadas constituye uno de los indicadores más utilizados para medir exactitud funcional y capacidad de resolución de problemas (13). En consecuencia, las pruebas unitarias automatizadas representan un instrumento fundamental dentro de investigaciones orientadas a evaluar calidad y desempeño del software generado automáticamente mediante inteligencia artificial.

2.2.7.2. SonarQube

SonarQube es una plataforma de análisis estático ampliamente utilizada para evaluar la calidad interna del software mediante la identificación automatizada de problemas estructurales presentes en el código fuente. Esta herramienta permite analizar diversos lenguajes de programación y generar métricas relacionadas con mantenibilidad, legibilidad y calidad del código, facilitando la detección temprana de deficiencias que pueden afectar la evolución y sostenibilidad de los sistemas software (14).

Dentro del ámbito de la ingeniería de software, SonarQube es utilizado para identificar code smells, entendidos como patrones de diseño o implementación que, sin constituir errores funcionales, pueden dificultar la comprensión, modificación y mantenimiento del código. Asimismo, la herramienta proporciona indicadores asociados a la calidad estructural del software, permitiendo detectar problemas relacionados con duplicación de código, complejidad excesiva y prácticas de programación que afectan negativamente la mantenibilidad del sistema (16).

En investigaciones relacionadas con generación automática de código mediante inteligencia artificial, SonarQube comenzó a emplearse como mecanismo de evaluación objetiva para analizar la calidad estructural de las soluciones producidas por modelos de lenguaje. Diversos estudios evidenciaron que el análisis automatizado facilita la identificación de diferencias entre distintas estrategias de generación de código y permite determinar el impacto de técnicas de Prompt Engineering sobre atributos relacionados con mantenibilidad y organización del software (12).

Asimismo, SonarQube incorpora métricas que contribuyen a la evaluación de la mantenibilidad, permitiendo analizar la facilidad con la que el código puede ser comprendido, corregido o modificado durante su ciclo de vida. Debido a ello, esta herramienta constituye un instrumento relevante para complementar la evaluación de calidad del software generado automáticamente, proporcionando información objetiva sobre la

estructura interna del código y apoyando la medición de atributos vinculados con mantenibilidad y complejidad (15).

2.2.7.3. Radon

Radon es una herramienta de análisis estático orientada principalmente a medir métricas relacionadas con complejidad ciclomática, mantenibilidad y tamaño del código fuente en proyectos desarrollados en Python. Esta herramienta permite calcular indicadores estructurales utilizados frecuentemente en investigaciones relacionadas con calidad de software y análisis automatizado de código (14). En el contexto de generación automática de código mediante inteligencia artificial, Radon comenzó a emplearse como mecanismo para evaluar la complejidad lógica de las soluciones generadas por modelos de lenguaje.

Investigaciones recientes demostraron que métricas como la complejidad ciclomática y el Maintainability Index permiten identificar diferencias significativas entre distintos enfoques de generación automática de código (15). Asimismo, estudios experimentales relacionados con Prompt Engineering evidenciaron que ciertas técnicas de prompting favorecen la generación de soluciones menos complejas y estructuralmente más comprensibles (12). En consecuencia, Radon constituye una herramienta importante para medir objetivamente atributos relacionados con mantenibilidad y complejidad del software generado automáticamente.

2.2.8. Benchmarks de evaluación

Los benchmarks y plataformas de evaluación corresponden a conjuntos estandarizados de problemas y métricas utilizados para medir el desempeño de modelos de inteligencia artificial en tareas específicas. Dentro del ámbito de la generación automática de código, estos mecanismos permiten evaluar de manera objetiva la capacidad de los modelos de lenguaje para resolver problemas algorítmicos, producir soluciones funcionales y generar software con adecuados niveles de calidad (17). En investigaciones relacionadas con Prompt Engineering y generación automática de software, el uso de benchmarks adquirió gran

importancia debido a que proporciona escenarios homogéneos para comparar distintas técnicas de prompting y diferentes modelos generativos.

Diversos estudios recientes utilizaron plataformas de programación competitiva y conjuntos de problemas estandarizados con el propósito de medir exactitud funcional, complejidad, mantenibilidad y desempeño del código generado automáticamente (12). Asimismo, investigaciones experimentales demostraron que la utilización de benchmarks facilita la reproducibilidad de resultados y permite validar objetivamente el impacto de las estrategias de interacción utilizadas con modelos de lenguaje (13). Debido a ello, los benchmarks se consolidaron como componentes fundamentales dentro de investigaciones orientadas a evaluar calidad de software generado mediante inteligencia artificial.

2.2.8.1. Leetcode

LeetCode es una plataforma de programación ampliamente utilizada para la resolución de problemas algorítmicos y preparación técnica en ingeniería de software. La plataforma contiene ejercicios clasificados por niveles de dificultad y categorías temáticas, permitiendo evaluar habilidades relacionadas con estructuras de datos, algoritmos y lógica computacional (17). Debido a la estandarización de sus problemas y casos de prueba, LeetCode comenzó a utilizarse frecuentemente como entorno experimental dentro de investigaciones relacionadas con generación automática de código mediante modelos de lenguaje.

Diversos estudios utilizaron problemas de LeetCode para comparar el rendimiento de distintos modelos generativos y técnicas de Prompt Engineering, debido a que la plataforma permite medir objetivamente exactitud funcional y eficiencia de las soluciones generadas (12). Asimismo, investigaciones recientes demostraron que la complejidad progresiva de los ejercicios facilita el análisis del comportamiento de los modelos frente a distintos niveles de dificultad computacional (15). En consecuencia, LeetCode constituye una plataforma adecuada para desarrollar investigaciones experimentales relacionadas con evaluación automática de código generado mediante inteligencia artificial.

2.2.8.2. HumanEval

HumanEval es un benchmark desarrollado para evaluar la capacidad de modelos de lenguaje en tareas de generación automática de código mediante pruebas funcionales automatizadas. Este conjunto de evaluación contiene problemas de programación acompañados por especificaciones y casos de prueba diseñados para verificar si las soluciones generadas cumplen correctamente los requisitos establecidos (12). HumanEval adquirió relevancia dentro del campo de inteligencia artificial aplicada al desarrollo de software debido a que permite medir exactitud funcional y razonamiento lógico de los modelos generativos de manera objetiva y reproducible.

Investigaciones recientes utilizaron HumanEval para comparar el desempeño de distintos modelos de lenguaje y técnicas de prompting, identificando diferencias significativas en la capacidad de resolución de problemas complejos (13). Asimismo, diversos estudios demostraron que métricas basadas en porcentaje de pruebas superadas constituyen indicadores efectivos para evaluar calidad funcional del código generado automáticamente (17). Debido a ello, HumanEval se consolidó como uno de los benchmarks más utilizados dentro de investigaciones relacionadas con generación automática de software mediante inteligencia artificial.

2.2.8.3. APPS

APPS (Automated Programming Progress Standard) es un benchmark orientado a evaluar capacidades de programación automática en modelos de lenguaje mediante problemas computacionales de distinta complejidad. Este conjunto de evaluación incluye ejercicios de programación competitiva, especificaciones textuales y casos de prueba automatizados que permiten medir exactitud funcional y capacidad de razonamiento algorítmico (13). A diferencia de otros benchmarks, APPS incorpora problemas más extensos y complejos, lo que facilita analizar el comportamiento de los modelos frente a tareas que requieren múltiples pasos lógicos y planificación estructurada.

Diversos estudios experimentales utilizaron APPS para evaluar el impacto de técnicas de Prompt Engineering sobre el desempeño de modelos generativos, evidenciando que estrategias como Chain-of-Thought prompting y few-shot prompting mejoran significativamente la capacidad de resolución de problemas (12). Asimismo, investigaciones recientes señalaron que APPS permite evaluar no solamente exactitud funcional, sino también robustez y coherencia del código generado automáticamente (15). En consecuencia, este benchmark constituye una herramienta relevante para investigaciones orientadas a comparar técnicas de estructuración de prompts y calidad de software generado mediante modelos de inteligencia artificial.

2.3. MarcoConceptual

- **Inteligencia Artificial Generativa:**

La inteligencia artificial generativa es una rama de la inteligencia artificial orientada a la creación automática de contenido mediante modelos entrenados con grandes volúmenes de datos. Estos sistemas son capaces de generar texto, imágenes, audio y código fuente a partir de instrucciones proporcionadas por los usuarios (12).

- **Modelos de Lenguaje de Gran Escala (LLMs):**

Los modelos de lenguaje de gran escala, conocidos como Large Language Models (LLMs), son sistemas basados en arquitecturas transformer entrenados para comprender y generar lenguaje natural. En el ámbito de la ingeniería de software, estos modelos son utilizados para tareas relacionadas con generación automática de código, documentación y asistencia al programador (13).

- **Prompt Engineering:**

El Prompt Engineering corresponde al conjunto de técnicas utilizadas para estructurar instrucciones dirigidas a modelos de inteligencia artificial con el propósito de mejorar la calidad y precisión de las respuestas generadas (15).

- **Zero-shot Prompting:**

El zero-shot prompting es una técnica de interacción con modelos de lenguaje donde el sistema recibe únicamente la descripción del problema sin ejemplos previos de resolución (12).

- **Few-shot Prompting:**

El few-shot prompting consiste en proporcionar ejemplos previos dentro del prompt con la finalidad de orientar al modelo hacia el tipo de respuesta esperada (13).

- **Chain-of-Thought Prompting:**

El Chain-of-Thought prompting es una estrategia de prompting basada en inducir razonamiento paso a paso antes de generar una solución final, mejorando la capacidad del modelo para resolver problemas complejos (17).

- **Role Prompting:**

El role prompting corresponde a una técnica mediante la cual se asigna un rol específico al modelo de lenguaje para influir sobre el estilo y comportamiento de las respuestas generadas (15).

- **Generación automática de código:**

La generación automática de código es el proceso mediante el cual modelos de inteligencia artificial producen fragmentos de software a partir de instrucciones escritas en lenguaje natural (16).

- **Calidad del software:**

La calidad del software hace referencia al grado en que un producto de software cumple requisitos funcionales y no funcionales relacionados con eficiencia, mantenibilidad, seguridad y confiabilidad (16).

- **Exactitud funcional:**

La exactitud funcional corresponde al grado en que un software ejecuta correctamente las funciones y requisitos para los cuales fue desarrollado (17).

- **Mantenibilidad:**

La mantenibilidad es la capacidad de un sistema software para ser comprendido, corregido o modificado de manera eficiente durante su ciclo de vida (14).

- **Complejidad:**

Grado de dificultad asociado a la comprensión, prueba y mantenimiento de un sistema software debido a su estructura lógica y organización interna (16).

- **Análisis estático de código:**

El análisis estático es una técnica de evaluación de software que permite detectar errores, vulnerabilidades y problemas de calidad sin necesidad de ejecutar el programa (14).

- **SonarQube:**

SonarQube es una plataforma de análisis estático utilizada para evaluar métricas relacionadas con mantenibilidad, vulnerabilidades y deuda técnica del software (12).

- **Radon:**

Radon es una herramienta de análisis estático especializada en medir complejidad ciclomática y mantenibilidad de código desarrollado en Python (15).

- **Pruebas unitarias automatizadas:**

Las pruebas unitarias automatizadas son procedimientos utilizados para validar automáticamente el funcionamiento correcto de funciones o componentes específicos de un software (17).

- **Maintainability Index:**

Métrica utilizada para estimar la facilidad de mantenimiento de un software a partir de características estructurales del código fuente, como complejidad, tamaño y legibilidad (14).

- **Complejidad Ciclomática:**

Métrica propuesta por McCabe que mide la cantidad de caminos lógicos independientes existentes dentro de un programa, permitiendo estimar su complejidad estructural (14).

- **Benchmark:**

Un benchmark es un conjunto estandarizado de problemas, pruebas o métricas utilizado para evaluar y comparar el desempeño de modelos, algoritmos o sistemas computacionales (12).

- **LeetCode:**

LeetCode es una plataforma de programación utilizada para resolver problemas algorítmicos y evaluar habilidades relacionadas con estructuras de datos y lógica computacional (17).

- **HumanEval:**

HumanEval es un benchmark diseñado para evaluar la capacidad de modelos de lenguaje en tareas de generación automática de código mediante pruebas funcionales automatizadas (12).

- **APPS:**

APPS es un benchmark orientado a medir capacidades de programación automática mediante problemas computacionales de distinta complejidad y casos de prueba automatizados (13).

CAPITULO III

METODOLOGÍA

3.1. Enfoque, Tipo, Alcance, Método y Diseño de la Investigación

3.1.1. Enfoque de la Investigación

La investigación tendrá un enfoque cuantitativo, debido a que se trabajará con datos numéricos y métricas obtenidas mediante herramientas automatizadas de evaluación de calidad de software (20). En este sentido, se recopilarán indicadores relacionados con complejidad ciclomática, mantenibilidad, deuda técnica, vulnerabilidades y otros aspectos medibles del código generado por el modelo de lenguaje de gran escala ChatGPT-4o. Posteriormente, los datos obtenidos serán analizados de manera estadística con la finalidad de comparar el comportamiento de las distintas técnicas de estructuración de prompts y determinar su influencia sobre la calidad del software generado automáticamente.

3.1.2. Tipo de Investigación

La presente investigación corresponde al tipo aplicada, debido a que busca generar una solución tecnológica orientada a evaluar el impacto de distintas técnicas de estructuración de prompts sobre la calidad del código generado mediante inteligencia artificial, el anterior concepto se obtiene gracias a las definiciones de Hernandez et al.(20). En ese sentido, el estudio no se limita únicamente a la generación de conocimiento teórico, sino que plantea el desarrollo de una plataforma automatizada capaz de integrar procesos de generación, análisis y comparación de código utilizando métricas cuantitativas relacionadas con calidad de software. Asimismo, la investigación pretende contribuir al fortalecimiento de prácticas modernas de ingeniería de software basadas en automatización y análisis experimental de modelos de lenguaje.

3.1.3. Alcance de la Investigación

El alcance de la investigación será explicativo, debido a que se pretende determinar la influencia que ejercen las distintas técnicas de estructuración de prompts sobre variables relacionadas con calidad del software generado por el modelo de lenguaje de gran escala ChatGPT-4o. En este contexto, el estudio buscará establecer

relaciones de causa y efecto(20) entre las estrategias de Prompt Engineering utilizadas y los resultados obtenidos en términos de mantenibilidad, complejidad, deuda técnica y vulnerabilidades detectadas mediante herramientas automatizadas de análisis estático. Asimismo, la investigación permitirá identificar qué técnicas producen mejores resultados dentro de procesos de generación automática de código.

3.1.4.Método de Investigación

La presente investigación se desarrollará bajo el método hipotético-deductivo, debido a que parte de la formulación de hipótesis relacionadas con la influencia de las técnicas de estructuración de prompts sobre la calidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o, las cuales serán sometidas a verificación mediante procedimientos experimentales y análisis cuantitativo. Este método permitirá establecer una relación lógica entre la teoría existente sobre Prompt Engineering, generación automática de código y calidad del software, con los resultados obtenidos durante la ejecución de las pruebas experimentales. Asimismo, la investigación partirá de fundamentos teóricos y antecedentes científicos previamente estudiados para plantear hipótesis específicas que posteriormente serán contrastadas mediante la aplicación de distintas técnicas de prompting sobre problemas estandarizados de programación(20). Los datos obtenidos serán analizados utilizando métricas objetivas relacionadas con mantenibilidad, complejidad ciclomática, deuda técnica, seguridad y exactitud funcional del código generado. En este sentido, el método hipotético-deductivo permitirá comprobar de manera sistemática y objetiva si las variaciones en las técnicas de estructuración de prompts producen diferencias significativas en la calidad del software generado por modelos de inteligencia artificial.

3.1.5.Diseño de la Investigación

El diseño de la investigación será cuasi-experimental de medidas repetidas, debido a que se manipulará deliberadamente la variable independiente correspondiente a las técnicas de estructuración de prompts, con la finalidad de observar sus efectos sobre las variables dependientes relacionadas con calidad del software(20). Para ello, se aplicarán distintas estrategias de Prompt Engineering sobre problemas de programación previamente definidos y posteriormente se evaluará el código

generado utilizando métricas cuantitativas obtenidas mediante herramientas automatizadas de análisis estático. Asimismo, el diseño experimental permitirá mantener condiciones homogéneas durante las pruebas y comparar objetivamente los resultados obtenidos entre las distintas técnicas evaluadas.

3.2. Población y Muestra

La unidad de análisis de la presente investigación estará conformada por fragmentos de código fuente generados mediante modelos de lenguaje de gran escala (LLMs) a partir de distintas técnicas de estructuración de prompts aplicadas sobre problemas estandarizados de programación obtenidos de plataformas como LeetCode y HackerRank. Asimismo, se considerarán como objeto de evaluación los resultados producidos por una plataforma automatizada propuesta, destinada a gestionar el proceso de generación, análisis y comparación del código mediante métricas de calidad relacionadas con exactitud funcional, mantenibilidad y complejidad. En ese sentido, el estudio se desarrollará dentro del campo de la ingeniería de software y Prompt Engineering, tomando como referencia herramientas de evaluación estática y estándares reconocidos internacionalmente(1)(8).

3.2.1.Población

La población de estudio estará conformada por problemas algorítmicos de programación susceptibles de ser resueltos mediante generación automática de código utilizando modelos de lenguaje de gran escala, disponibles en plataformas de evaluación y práctica de programación como LeetCode y HackerRank. Dichos problemas constituyen unidades experimentales apropiadas para analizar el comportamiento de distintas técnicas de Prompt Engineering aplicadas a la generación automática de soluciones computacionales, permitiendo evaluar atributos relacionados con mantenibilidad, complejidad ciclomática, deuda técnica, seguridad y exactitud funcional del código generado (9). Asimismo, la población incluirá las distintas respuestas generadas por los modelos de inteligencia artificial bajo estrategias de Prompt Engineering como zero-shot, few-shot, chain-of-thought y role prompting, permitiendo analizar comparativamente el comportamiento del código producido en función de métricas relacionadas con exactitud funcional, mantenibilidad y complejidad.

3.2.2.Muestra

La muestra estará constituida por un subconjunto de problemas algorítmicos seleccionados de manera no probabilística e intencional a partir de plataformas como LeetCode y HackerRank, considerando criterios relacionados con nivel de dificultad, compatibilidad con el lenguaje de programación seleccionado, disponibilidad de casos de prueba automatizados y pertinencia para el análisis experimental. Sobre cada problema se aplicarán distintas técnicas de Prompt Engineering (zero-shot, few-shot, chain-of-thought y role prompting), generándose fragmentos de código que serán posteriormente evaluados mediante herramientas automatizadas de análisis estático y pruebas funcionales. (1).

3.3. Materiales y Métodos

3.3.1.Materiales

3.3.1.1. Recursos de hardware

Para el desarrollo de la investigación se utilizaron recursos de hardware destinados a la construcción, ejecución y evaluación experimental de la plataforma desarrollada. Entre los principales componentes utilizados se encontraron los siguientes:

- Procesador AMD Ryzen 5 3600.
- Memoria RAM de 16 GB.
- Unidad de almacenamiento SSD.
- Conexión a Internet para consumo de APIs de inteligencia artificial.
- Equipo de cómputo utilizado para desarrollo y pruebas experimentales.

3.3.1.2. Recursos de software

Los recursos de software empleados permitieron desarrollar, ejecutar y evaluar la plataforma experimental utilizada durante la investigación. Entre las principales herramientas utilizadas se consideraron:

- Sistema operativo Windows 10.
- Lenguaje de programación Python.
- Entornos virtuales para gestión de dependencias.
- Gestores de paquetes compatibles con Python.
- Librerías para consumo de APIs y procesamiento de datos.

3.3.1.3. Modelos de lenguaje utilizados

La generación automática de código fuente se realizó mediante modelos de lenguaje de gran escala accesibles a través de APIs externas. Para ello se utilizaron los siguientes recursos:

- APIs de modelos de inteligencia artificial.
- Consumo de tokens para generación automática de código.
- Solicitudes HTTP para comunicación con modelos generativos.
- Configuración de prompts para evaluación experimental.

3.3.1.4. Frameworks y librerías de desarrollo

Para la implementación de la plataforma experimental se utilizaron frameworks y librerías orientadas al desarrollo backend y automatización de procesos. Entre las principales tecnologías utilizadas se encontraron:

- Librerías para integración con APIs.
- Herramientas para manipulación de archivos JSON y CSV.
- Componentes frontend básicos para interacción local.
- Librerías para automatización de pruebas y procesamiento de resultados.

3.3.1.5. Herramientas de análisis y evaluación de código

Con el propósito de evaluar las variables dependientes de la investigación se utilizaron herramientas de análisis estático y validación automatizada de software. Entre las principales herramientas consideradas se encontraron:

- SonarQube para análisis de calidad y deuda técnica.
- Radon para medición de complejidad ciclomática y mantenibilidad.
- Bandit para detección de vulnerabilidades de seguridad.
- Pruebas automatizadas para validación funcional del código generado.

3.3.1.6. Base de datos y almacenamiento

El almacenamiento de información experimental se realizó mediante mecanismos ligeros orientados al registro y procesamiento de resultados. Para ello se utilizaron los siguientes recursos:

- Archivos JSON para almacenamiento estructurado.
- Archivos CSV para exportación y análisis de resultados.
- Registro de prompts y respuestas generadas.
- Almacenamiento de métricas y resultados experimentales.

3.3.1.7. Infraestructura y despliegue

La plataforma experimental fue implementada en un entorno local utilizando una arquitectura cliente-servidor simplificada. Para ello se emplearon los siguientes componentes:

- Entorno de ejecución local.
- Consumo remoto de APIs de inteligencia artificial.
- Arquitectura cliente-servidor local.
- Integración mediante conexión a Internet.

3.3.1.8. Herramientas de desarrollo y colaboración

Las actividades de desarrollo y trabajo colaborativo fueron apoyadas mediante herramientas orientadas al control de versiones y organización del proyecto. Entre las principales herramientas utilizadas se encontraron:

- Visual Studio Code como entorno de desarrollo.
- Git para control de versiones.

- GitHub para repositorio colaborativo del proyecto.

3.3.1.9. Dataset y plataformas de evaluación

Para el desarrollo de pruebas experimentales y validación de resultados se utilizaron problemas algorítmicos y plataformas de benchmarking reconocidas dentro del área de ingeniería de software. Entre los principales recursos utilizados se consideraron:

- Problemas algorítmicos de LeetCode.
- Casos de prueba automatizados.
- Benchmarks para evaluación de exactitud funcional.
- Problemas orientados a generación automática de código.

3.3.2. Métodos

La presente investigación utilizó un enfoque metodológico orientado al desarrollo experimental de una plataforma de evaluación de técnicas de Prompt Engineering aplicadas a generación automática de código mediante modelos de lenguaje de gran escala. Debido a que el proyecto involucró procesos de análisis, diseño, implementación, pruebas y validación de software, se adoptó una metodología de desarrollo ágil con características iterativas e incrementales, permitiendo construir progresivamente los componentes de la plataforma experimental y realizar ajustes continuos durante las fases de evaluación. Asimismo, se integraron técnicas propias de ingeniería de software relacionadas con análisis de requerimientos, diseño arquitectónico, automatización de pruebas y validación experimental de resultados.

3.3.2.1. Metodología de desarrollo de software

Para el desarrollo de la plataforma experimental se adoptó una metodología ágil debido a la naturaleza iterativa de la investigación y a la necesidad de realizar ajustes progresivos sobre prompts, herramientas de evaluación y componentes de integración con APIs de inteligencia artificial. Este enfoque permitió desarrollar funcionalidades incrementales, validar resultados de manera continua y adaptar el sistema según las necesidades identificadas durante la investigación.

Entre las principales características metodológicas consideradas se encontraron:

- Desarrollo iterativo e incremental.
- Integración continua de funcionalidades.
- Validación progresiva de resultados experimentales.
- Retroalimentación constante entre desarrollo y pruebas.
- Flexibilidad para modificar prompts y métricas de evaluación.

3.3.2.2. Técnicas de análisis y diseño

Durante la etapa de análisis se utilizaron técnicas orientadas a identificar requerimientos funcionales y no funcionales de la plataforma experimental. Asimismo, se definieron los componentes necesarios para integrar modelos de lenguaje, almacenamiento de resultados y herramientas de evaluación automatizada.

Entre las principales técnicas utilizadas se consideraron:

- Análisis de requerimientos funcionales.
- Identificación de variables e indicadores experimentales.
- Diseño de arquitectura cliente-servidor local.
- Modelado de flujo de procesamiento experimental.
- Diseño modular de componentes de evaluación.

3.3.2.3. Técnicas de desarrollo e implementación

La implementación de la plataforma experimental se realizó mediante prácticas orientadas al desarrollo modular y reutilización de componentes de software. Asimismo, se utilizaron técnicas de integración con APIs externas para comunicación con modelos de lenguaje de gran escala.

Entre las principales actividades desarrolladas se encontraron:

- Desarrollo backend para automatización de procesos.
- Integración con APIs de inteligencia artificial.

- Implementación de módulos de generación automática de código.
- Registro automatizado de prompts y respuestas generadas.
- Implementación de módulos de evaluación y análisis de métricas.

3.3.2.4. Técnicas de pruebas y validación

Con la finalidad de verificar el funcionamiento correcto de la plataforma y evaluar la calidad del código generado automáticamente, se utilizaron técnicas de pruebas funcionales y análisis automatizado de software. Estas técnicas permitieron medir el impacto de distintas estrategias de Prompt Engineering sobre las variables dependientes de la investigación.

Entre las principales técnicas de validación utilizadas se consideraron:

- Pruebas funcionales automatizadas.
- Análisis estático de código.
- Validación de exactitud funcional mediante casos de prueba.
- Comparación experimental entre técnicas de prompting.
- Evaluación de métricas de mantenibilidad y complejidad.

3.3.2.5. Arquitectura e infraestructura de implementación

La plataforma experimental fue diseñada utilizando una arquitectura cliente-servidor local orientada al consumo remoto de APIs de inteligencia artificial. Esta arquitectura permitió separar los módulos de interacción con modelos generativos, procesamiento de resultados y evaluación automatizada de código.

Los principales componentes arquitectónicos considerados fueron:

- Módulo de gestión de prompts.
- Módulo de integración con APIs de inteligencia artificial.
- Módulo de almacenamiento de resultados experimentales.
- Módulo de evaluación automática de código.
- Interfaz local para interacción y visualización de resultados.

3.3.2.6. Fases del proyecto

Fase 1: Fundamentación teórica

Durante esta fase se realizó la revisión de la literatura relacionada con Prompt Engineering, modelos de lenguaje de gran escala y calidad del software. Asimismo, se definieron los fundamentos teóricos, variables, dimensiones e indicadores que sustentan la investigación.

Actividades principales:

- Revisión bibliográfica.
- Elaboración del marco teórico y conceptual.
- Definición de variables, dimensiones e indicadores.
- Formulación de objetivos e hipótesis.
- Diseño metodológico de la investigación.

Fase 2: Diseño experimental

En esta etapa se definieron los elementos necesarios para la ejecución de los experimentos, incluyendo la selección de problemas algorítmicos, técnicas de Prompt Engineering y criterios de evaluación de calidad del código generado.

Actividades principales:

- Selección de problemas de LeetCode.
- Definición de técnicas de Prompt Engineering.
- Diseño del procedimiento experimental.
- Definición de métricas e instrumentos de medición.
- Planificación de las pruebas experimentales.

Fase 3: Desarrollo de la plataforma experimental

Durante esta fase se implementó la plataforma utilizada para automatizar la generación, almacenamiento y evaluación del código producido por los modelos de lenguaje.

Actividades principales:

- Diseño de la arquitectura de la plataforma.
- Desarrollo del backend.
- Desarrollo del frontend.
- Integración de herramientas de análisis de código.
- Implementación de módulos de almacenamiento y procesamiento.
- Pruebas de funcionamiento de la plataforma.

Fase 4: Ejecución experimental

En esta etapa se ejecutaron los experimentos definidos en el diseño metodológico, generando código mediante las distintas técnicas de Prompt Engineering y obteniendo las métricas necesarias para el análisis.

Actividades principales:

- Construcción de prompts experimentales.
- Generación automática de código.
- Ejecución de pruebas unitarias automatizadas.
- Obtención de métricas de calidad del software.
- Consolidación de datos experimentales.

Fase 5: Análisis de resultados

Durante esta fase se procesaron los datos recolectados para identificar diferencias entre las técnicas de Prompt Engineering evaluadas y determinar su impacto sobre la calidad del código generado.

Actividades principales:

- Procesamiento estadístico de datos.

- Análisis descriptivo e inferencial.
- Interpretación de resultados.
- Contraste de hipótesis.

Fase 6: Elaboración del informe final

Finalmente, se documentaron los resultados obtenidos y se elaboró el informe final de investigación.

Actividades principales:

- Redacción del capítulo de resultados.
- Elaboración de conclusiones y recomendaciones.
- Identificación de limitaciones y trabajos futuros.
- Correcciones finales.
- Entrega del informe final.

3.4. Técnicas de Muestreo

La presente investigación empleó un muestreo no probabilístico de tipo intencional o por criterio del investigador. Según Hernández Sampieri, Fernández Collado y Baptista, en el muestreo no probabilístico la selección de los elementos depende de las características y criterios definidos por el investigador, en función de los objetivos del estudio y no de la probabilidad estadística de selección (20).

En este contexto, la muestra estuvo conformada por treinta (30) problemas algorítmicos seleccionados de manera intencional a partir de las plataformas LeetCode y HackerRank, distribuidos en tres niveles de dificultad: diez (10) problemas de nivel fácil, diez (10) de nivel intermedio y diez (10) de nivel difícil. La selección se realizó considerando criterios de pertinencia para la generación automática de código, disponibilidad de casos de prueba automatizados, compatibilidad con el lenguaje de programación Python y adecuación para la evaluación experimental de las técnicas de Prompt Engineering.

Sobre cada problema seleccionado se aplicaron cuatro tratamientos experimentales correspondientes a las técnicas de estructuración de prompts: Zero-Shot Prompting, Few-Shot Prompting, Chain-of-Thought Prompting y Role Prompting. Cada técnica generó

una solución de código fuente mediante el modelo de lenguaje ChatGPT-4o bajo condiciones controladas, permitiendo comparar objetivamente su influencia sobre la calidad del código generado.

3.5. Técnicas e Instrumentos de Recolección de Datos

3.5.1. Técnicas de Recolección de Datos

La investigación utilizó la técnica de análisis documental automatizado para la recolección de datos. Esta técnica consistió en el análisis sistemático de los fragmentos de código fuente generados por el modelo de lenguaje ChatGPT-4o a partir de las distintas técnicas de estructuración de prompts aplicadas sobre los problemas seleccionados.

De acuerdo con Hernández Sampieri et al., la recolección de datos en estudios cuantitativos implica obtener información que permita medir las variables de investigación mediante procedimientos objetivos y sistemáticos (20). En ese sentido, la información fue obtenida directamente a partir de los resultados generados por el experimento y procesada mediante herramientas automatizadas de evaluación de software.

Para la dimensión exactitud funcional se emplearon pruebas unitarias automatizadas que permitieron determinar el porcentaje de casos de prueba superados por cada solución generada. Para la dimensión mantenibilidad se utilizaron métricas obtenidas mediante la herramienta SonarQube, específicamente el Maintainability Index y el Technical Debt Ratio. Finalmente, para la dimensión complejidad se utilizó la herramienta Radon, encargada de calcular la complejidad ciclomática del código fuente generado.

3.5.2. Instrumento de Recolección de Datos

Como instrumento de recolección de datos se utilizó una Ficha de Registro Experimental, diseñada específicamente para registrar de forma estructurada los resultados obtenidos durante la ejecución de los tratamientos experimentales.

La ficha permitió registrar información relacionada con la identificación del experimento, problema evaluado, técnica de prompting aplicada y métricas

asociadas a las dimensiones de la variable dependiente: exactitud funcional, mantenibilidad y complejidad. Asimismo, incorporó indicadores cuantitativos obtenidos automáticamente mediante herramientas de análisis estático y pruebas funcionales, garantizando objetividad y uniformidad en el proceso de medición.

- El instrumento estuvo conformado por tres dimensiones de evaluación:
- Exactitud funcional: porcentaje de pruebas unitarias superadas.
- Mantenibilidad: Maintainability Index (MI) y Technical Debt Ratio (TDR).
- Complejidad: Complejidad Ciclomática (CC).

La información consignada en la ficha fue generada automáticamente por la plataforma desarrollada para la investigación, minimizando errores de registro y asegurando la trazabilidad de los resultados experimentales.

3.6. Validez y Confiabilidad

3.6.1. Validez

La validez de un instrumento se refiere al grado en que este mide realmente la variable que pretende medir. Según Hernández Sampieri, Fernández Collado y Baptista, la validez hace referencia al grado en que un instrumento refleja adecuadamente el concepto o variable que se desea evaluar (20).

Para determinar la validez de contenido de la Ficha de Registro Experimental, el instrumento fue sometido a juicio de expertos. Este procedimiento consistió en la evaluación de la pertinencia, claridad, coherencia y relevancia de los indicadores considerados para medir la calidad del código generado por el modelo de lenguaje ChatGPT-4o. Los especialistas seleccionados poseen experiencia en ingeniería de software, calidad de software, inteligencia artificial e investigación científica.

Posteriormente, las valoraciones emitidas por los expertos fueron procesadas mediante el coeficiente V de Aiken, el cual permite cuantificar el grado de acuerdo entre los jueces respecto a la validez de los elementos incluidos en un instrumento de medición. Este coeficiente es ampliamente utilizado para evaluar la validez de contenido en investigaciones cuantitativas.

La V de Aiken se calculó mediante la siguiente expresión:

$$V = \sum s / [n(c - 1)]$$

Donde:

- V: coeficiente de validez de Aiken.
- s: puntuación asignada por cada juez menos la puntuación mínima de la escala utilizada.
- n: número de jueces expertos.
- c: número de categorías de valoración.

Los resultados obtenidos permitieron determinar el nivel de validez de contenido de la Ficha de Registro Experimental, garantizando que los indicadores seleccionados representan adecuadamente las dimensiones de exactitud funcional, mantenibilidad y complejidad consideradas en la investigación.

Se tiene por lo mismo la tabla que representa el calculo anterior:

TABLA 1. Calculo de la Validez por V de Aiken

Nº	Indicadores de Validación	Experto 1(L. Sota)	Experto 2(N. Gutiérrez)	Experto 3(J. Mendoza)	Promedio por Indicador
1	Claridad	90%	90%	90%	90.00%
2	Objetividad	90%	100%	85%	91.67%
3	Actualidad	95%	95%	90%	93.33%
4	Organización	90%	90%	90%	90.00%
5	Suficiencia	85%	85%	90%	86.67%
6	Intencionalidad	90%	95%	85%	90.00%
7	Consistencia	90%	90%	90%	90.00%
8	Coherencia	95%	90%	90%	91.67%
9	Metodología	85%	100%	90%	91.67%
10	Conveniencia	90%	85%	80%	85.00%
	PROMEDIO TOTAL	89.00%	92.00%	88.00%	90.00%

Las validaciones correspondientes para la elaboración de esta tabla se encuentran en los Anexos G, H e I.

3.6.2.Confiabilidad

La confiabilidad se refiere al grado en que un instrumento produce resultados consistentes y estables cuando se aplica repetidamente bajo las mismas

condiciones. Hernández Sampieri, Fernández Collado y Baptista señalan que un instrumento es confiable cuando genera mediciones similares al aplicarse en diferentes momentos a los mismos sujetos u objetos de estudio, siempre que las condiciones permanezcan constantes (20).

Para determinar la confiabilidad de la Ficha de Registro Experimental se empleará el método Test-Retest. Este procedimiento consiste en aplicar el instrumento en dos momentos diferentes sobre una misma muestra de elementos, manteniendo constantes las condiciones de evaluación, con el propósito de verificar la estabilidad de las mediciones obtenidas.

En la presente investigación, la prueba se realizará utilizando un subconjunto de problemas de programación seleccionados de la muestra experimental. Cada problema será procesado nuevamente mediante las mismas técnicas de estructuración de prompts, conservando los parámetros de control establecidos: modelo ChatGPT-4o, temperatura de generación igual a 0.0, lenguaje de programación Python y condiciones idénticas de ejecución.

Posteriormente, los resultados obtenidos en ambas mediciones serán comparados mediante el coeficiente de correlación de Pearson, con el fin de determinar el grado de estabilidad temporal de las métricas registradas. Un coeficiente elevado indicará que el instrumento presenta un nivel adecuado de confiabilidad y que las mediciones obtenidas son consistentes y reproducibles.

Asimismo, debido a que los datos son registrados automáticamente por la plataforma experimental mediante herramientas de análisis estático y pruebas unitarias automatizadas, se minimiza la intervención humana durante el proceso de medición, contribuyendo a la estabilidad y consistencia de los resultados obtenidos.

CAPITULO IV

ASPECTOS ADMINISTRATIVOS

4.1. Presupuesto

TABLA 2. Tabla de Presupuesto de la Investigación

Macro categoría	Ítem	Cantidad	Unidad	Precio unitario (S/.)	Total (S/.)
Recursos Humanos	Investigadores tesisistas	3	Personas	S/2000.00	S/6000.00
	Asesoría metodológica ocasional	1	Servicio	S/ 150.00	S/150.00
Equipamiento	Computadora Ryzen 5 3600	1	Equipo	S/3000.00	S/3000.00
	Memoria RAM 16 GB (incluida)	1	Componente	S/0.00	S/0.00
	Conexión a Internet	6	Meses	S/80.00	S/ 480.00
Software	Visual Studio Code	1	Licencia	S/ 0.00	S/0.00
	GitHub	1	Servicio	S/0.00	S/0.00
	Python	1	Software	S/0.00	S/0.00
	SonarQube Community Edition	1	Software	S/0.00	S/0.00
	Radon	1	Librería	S/0.00	S/0.00
	Bandit	1	Librería	S/0.00	S/0.00
Servicios	Consumo de API de IA (tokens)	1	Servicio	S/350.00	S/350.00
	Electricidad y consumo energético	6	Meses	S/40.00	S/240.00
Otros	Impresiones y documentación	1	Lote	S/120.00	S/120.00
	Contingencias técnicas	1	Fondo	S/150.00	S/150.00
				Total	S/10,490.00

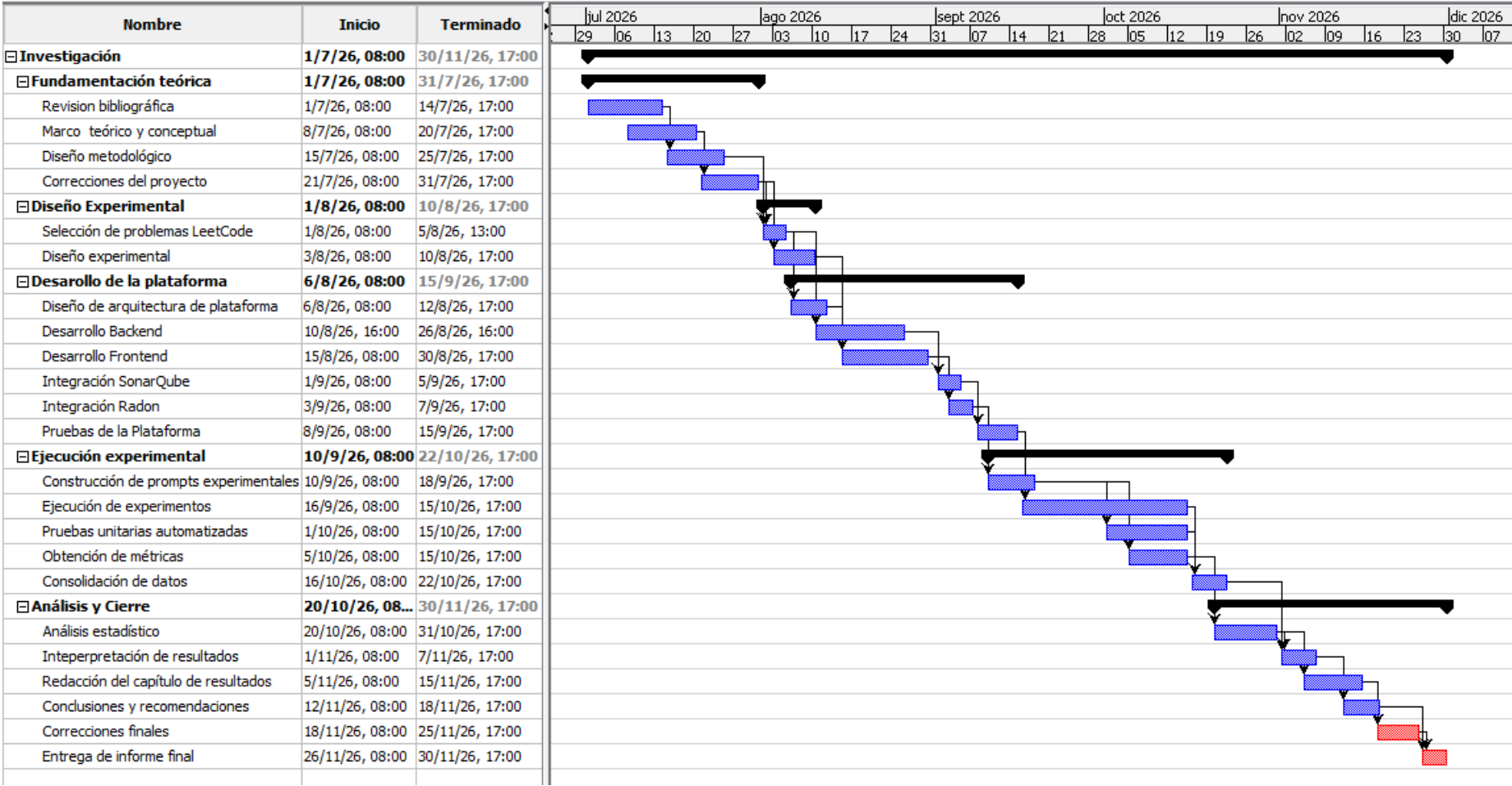
4.2. Cronograma

El cronograma de actividades permitió planificar y organizar temporalmente las tareas necesarias para el desarrollo de la presente investigación y la implementación de la plataforma experimental. Su elaboración facilitó la distribución ordenada de las actividades relacionadas con la revisión bibliográfica, construcción del marco teórico y conceptual, definición metodológica, desarrollo de la plataforma de software, ejecución de los experimentos y análisis de los resultados obtenidos.

La planificación temporal se estableció para el periodo comprendido entre julio y noviembre de 2026. Durante este periodo se programaron actividades de investigación documental, diseño experimental, desarrollo de la plataforma web, integración de herramientas de análisis de código, construcción de prompts experimentales, ejecución de pruebas automatizadas, obtención de métricas de calidad del software y análisis estadístico de los datos recolectados. Asimismo, se contemplaron actividades destinadas a la interpretación de resultados, redacción del informe final, formulación de conclusiones y recomendaciones, y realización de correcciones finales previas a la entrega de la investigación.

La representación gráfica del cronograma se desarrolló mediante un diagrama de Gantt elaborado en ProjectLibre, permitiendo visualizar la secuencia, duración y superposición de las actividades programadas. Del mismo modo, la planificación consideró un calendario de trabajo continuo durante toda la semana, incluyendo actividades realizadas los días sábados y domingos, con el propósito de optimizar el tiempo disponible y asegurar el cumplimiento de los plazos establecidos para la investigación.

FIGURA1. Diagrama de Gantt del Cronograma de Actividades



REFERENCIAS

1. HOU, Xinyi, ZHAO, Yanjie, LIU, Yue, YANG, Zhou, WANG, Kailong, LI, Li, LUO, Xiapu, LO, David, GRUNDY, John and WANG, Haoyu. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology*. Online. November 2024. Vol. 33, no. 8, p. 1–79. DOI 10.1145/3695988. Available from: <https://dl.acm.org/doi/epdf/10.1145/3695988>
2. LIU, Yue, LE-CONG, Thanh, WIDYASARI, Ratnadira, TANTITHAMTHAVORN, Chakkrit, LI, Li, LE, Xuan-Bach D and LO, David. Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues. *ACM Transactions on Software Engineering and Methodology*. Online. June 2024. Vol. 33, no. 5, p. 1–26. DOI 10.1145/3643674. Available from: <https://dl.acm.org/doi/epdf/10.1145/3643674>
3. ASFANDYAR, Muhammad Waseem Akhtar. EVALUATING THE IMPACT OF AI CODE GENERATED OF SOFTWARE QUALITY AND OBSERVED STUDY OF MAINTAINABILITY, SECURITY AND PERFORMANCE. *Spectrum of Engineering Sciences*. Online. December 2025. Vol. 3, no. 12, p. 1379–1389. DOI 10.5281/ZENODO.18172279. Available from: <https://thesesjournal.com/index.php/1/article/download/1810/1360>
4. ALJOHANI, Ahmed and DO, Hyunsook. PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects. In : *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. Online. Istanbul Turkiye : ACM, June 2025. p. 371–382. ISBN 979-8-4007-1385-9. DOI 10.1145/3756681.3756976. Available from: <https://dl.acm.org/doi/epdf/10.1145/3756681.3756976>
5. GOBIERNO DEL PERÚ. *Estrategia Nacional de Inteligencia Artificial 2021-2026: Informe de Progreso y Actualización* Online. Lima, Perú, 2024. [Accessed 21 June 2026]. Available from: https://cdn.www.gob.pe/uploads/document/file/8686097/7190386-propuesta_-estrategia-nacional-de-inteligencia-artificial.pdf?v=1759424094

6. PAZ FERNÁNDEZ, Crishtian Brenon, DIAZ SIFUENTES, Sergio Helí and TORRES VILLANUEVA, Marcelino. Optimización de Modelos de Lenguaje Grande (LLMs) a través del Prompt Engineering. *Innovation and Software*. Online. September 2025. Vol. 6, no. 2, p. 6–11. DOI 10.48168/innosoft.s24.a212. Available from: <https://revistas.ulasalle.edu.pe/innosoft/article/view/212/381>

7. GOBIERNO REGIONAL DE CUSCO. *Plan de Gobierno Digital 2025-2027* Online. Cusco, Perú, 2024. [Accessed 21 June 2026]. Available from: [https://transparencia.regioncusco.gob.pe/transparencia/planeamiento_organizacion/gobiernodigital/PLAN%20DE%20GOBIERNO%20DIGITAL%202025-2027%20GORE%20CUSCO%20-\[VB\]\[VB\].pdf](https://transparencia.regioncusco.gob.pe/transparencia/planeamiento_organizacion/gobiernodigital/PLAN%20DE%20GOBIERNO%20DIGITAL%202025-2027%20GORE%20CUSCO%20-[VB][VB].pdf)

8. ISO/IEC. *ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Models*. Online. International Organization for Standardization, 2011. [Accessed 21 June 2026]. Available from: <https://www.iso.org/standard/35733.html>

9. CORDEIRO, Jonathan, NOEI, Shayan and ZOU, Ying. An Empirical Study on the Code Refactoring Capability of Large Language Models. *ACM Transactions on Software Engineering and Methodology*. Online. March 2026. P. 3801158. DOI 10.1145/3801158. Available from: <https://dl.acm.org/doi/epdf/10.1145/3801158>

10. VASWANI, Ashish, BRAIN, Google, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N, KAISER, Łukasz and POLOSUKHIN, Illia. *Attention Is All You Need* Online. [no date]. [Accessed 21 June 2026]. Available from: <https://arxiv.org/abs/1706.03762>

11. ZHAO, Wayne Xin, ZHOU, Kun, LI, Junyi, TANG, Tianyi, WANG, Xiaolei, HOU, Yupeng, MIN, Yingqian, ZHANG, Beichen, ZHANG, Junjie, DONG, Zican, DU, Yifan, YANG, Chen, CHEN, Yushuo, CHEN, Zhipeng, JIANG, Jinhao, REN, Ruiyang, LI, Yifan, TANG, Xinyu, LIU, Zikang, LIU, Peiyu, NIE, Jian-Yun and WEN, Ji-Rong. A Survey of Large Language Models. Online. 18 March 2026. Available from: <http://arxiv.org/abs/2303.18223>

12. KHOJAH, Ranim, DE OLIVEIRA NETO, Francisco Gomes, MOHAMAD, Mazen and LEITNER, Philipp. The Impact of Prompt Programming on Function-

- Level Code Generation. *IEEE Transactions on Software Engineering*. Online. 2025. Vol. 51, no. 8, p. 2381–2395. [Accessed 21 June 2026]. DOI 10.1109/TSE.2025.3587794. Available from: <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=11077752>
13. GARCIA, Enio, JUNIOR, Santana, BENJAMIN, Gabriel, ARAUJO, Melissa, SANTOS, Harrison, FREITAS, David, ALMEIDA, Eduardo, ANSELMO DA MOTA, Paulo, NETO, Silveira, LI, Jiawei, CHUN, Jina and AHMED, Iftekhar. *Which Prompting Technique Should I Use? An Empirical Investigation of Prompting Techniques for Software Engineering Tasks* Online. 2025. [Accessed 21 June 2026]. Available from: <https://github.com/prompt-study/prompt-tasks-study/tree/main> <https://arxiv.org/abs/2506.05614>
 14. GHOSH PAUL, Debalina, ZHU, Hong and BAYLEY, Ian. *Investigating The Smells of LLM Generated Code* ★ Online. 2025. [Accessed 21 June 2026]. Available from: <https://arxiv.org/abs/2510.03029>
 15. FANYI ZHAO, MINGZHUO YU and CHUANKAI LUO. A Comparative Empirical Study of Prompting Strategies for Code Generation with Large Language Models. *Journal of Advanced Computing Systems*. Online. 9 December 2025. Vol. 5, no. 12, p. 26–37. DOI 10.69987/JACS.2025.51203. Available from: <https://scipublication.com/index.php/JACS/article/view/362/329>
 16. LI, Ruimeng, MO, Ran, ZENG, Yao, LIU, Zili, LI, Zengyang and MA, Yutao. A Multi-Dimensional Quality Evaluation of Code Generated by LLM-Based Tools Across Multiple Programming Languages. *ACM Transactions on Software Engineering and Methodology*. Online. 5 March 2026. DOI 10.1145/3800582. Available from: <https://dl.acm.org/doi/epdf/10.1145/3800582>
 17. COIGNION, Tristan, QUINTON, Clément and ROUVOY, Romain. A Performance Study of LLM-Generated Code on Leetcode. In : *ACM International Conference Proceeding Series*. Online. Association for Computing Machinery, 18 June 2024. p. 79–89. [Accessed 21 June 2026]. ISBN 9798400717017. DOI 10.1145/3661167.3661221. Available from: <https://dl.acm.org/doi/epdf/10.1145/3661167.3661221>

18. AZEEM, Samina, NAVEED, Muhammad Shumail, SAJID, Muhammad and ALI, Imran. AI vs. Human Programmers: Complexity and Performance in Code Generation. *VAWKUM Transactions on Computer Sciences*. Online. May 2025. Vol. 13, no. 1, p. 201–216. DOI 10.21015/vtcs.v13i1.2043. Available from: <https://vfast.org/journals/index.php/VTCS/article/view/2043/1695>
19. PARDO MESIAS, Samuel Ricardo. *Mantenibilidad de productos de software según el modelo SQUARE ISO/IEC 25000*. Online. 2018. Tingo María, Perú. [Accessed 21 June 2026]. Available from: <https://repositorio.unas.edu.pe/items/3b727fc4-71d9-438f-b418-df29f40b41a8>
20. HERNÁNDEZ SAMPIERI, Roberto, FERNÁNDEZ COLLADO, Carlos and BAPTISTA LUCIO, María del Pilar. *Metodología de la Investigación*. Online. 6. México D.F. : McGraw-Hill / Interamericana Editores, S.A. de C.V., 2014. [Accessed 21 June 2026]. ISBN 978-1-4562-2396-0. Available from: <https://www.esup.edu.pe/wp-content/uploads/2020/12/2.%20Hernandez,%20Fernandez%20y%20Baptista-Metodolog%C3%ADa%20Investigacion%20Cientifica%206ta%20ed.pdf>

ANEXOS

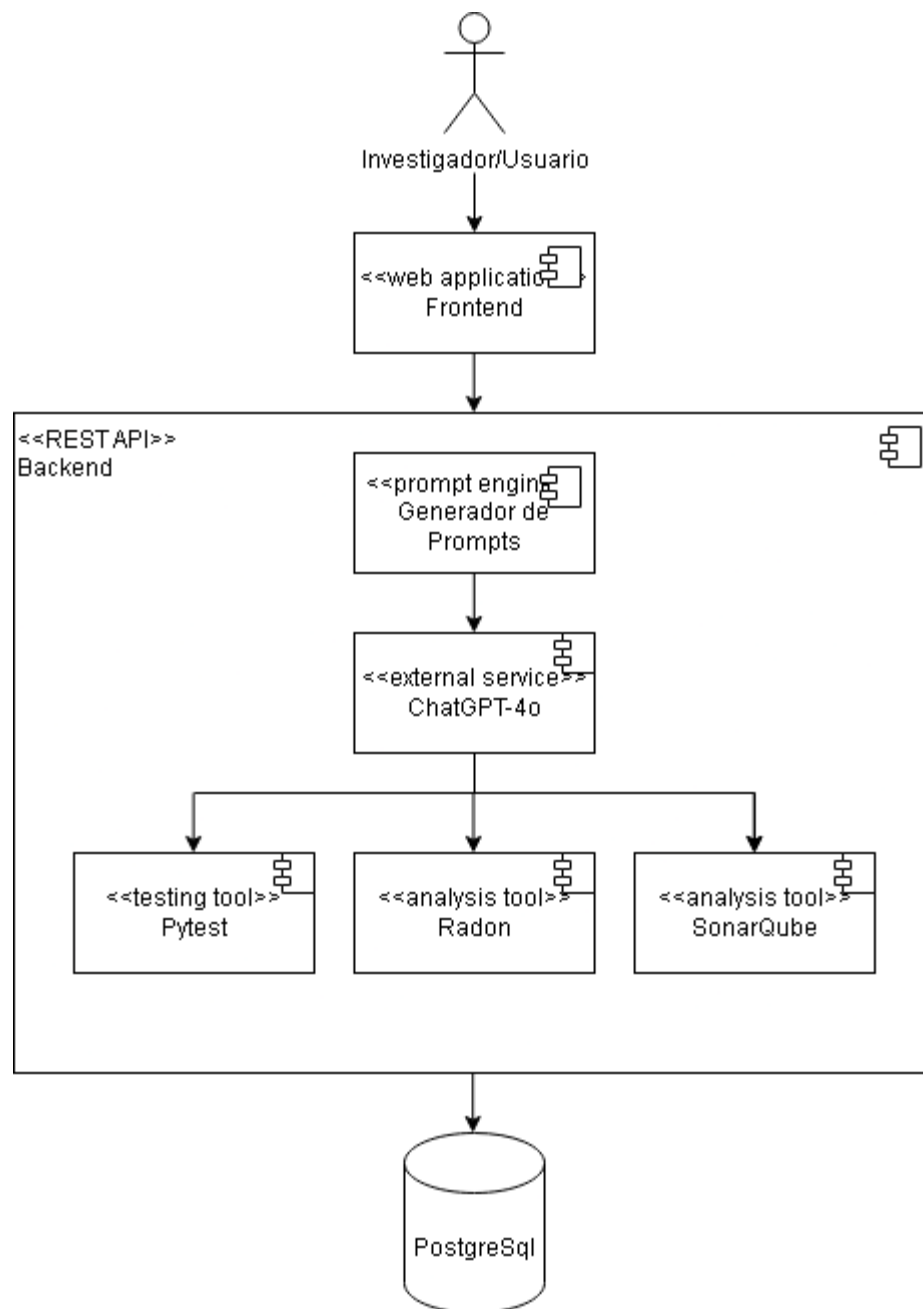
Anexo A. Matriz de Consistencia

Problema de Investigación	Objetivo de Investigación	Hipótesis de Investigación	Variables	Metodología
¿Cómo influyen las técnicas de estructuración de prompts en la calidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o?	Determinar la influencia de las técnicas de estructuración de prompts en la calidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.	Las técnicas de estructuración de prompts influyen significativa y positivamente en la calidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.	Variable Independiente: Técnicas de prompts Tratamientos: • Zero-Shot prompting • Few-Shot prompting • Chain-of-Thought prompting • Role prompting Variable Dependiente: Calidad del Código Generado Dimensiones: • Exactitud funcional • Mantenibilidad • Complejidad	Enfoque: Cuantitativo Tipo: Aplicada Alcance: Explicativo Método: Hipotético-deductivo Diseño: Cuasi-experimental con medidas repetidas Población: Problemas de programación de LeetCode/HackerRank Muestra: Subconjunto no probabilístico e intencional Técnica: Análisis documental Instrumentos: Ficha de registro experimental
Problemas Específicos	Objetivos Específicos	Hipótesis Específicas		
PE1. ¿Cómo influyen las técnicas de estructuración de prompts en la exactitud funcional del código generado por el modelo de lenguaje de gran escala ChatGPT-4o?	OE1. Determinar la influencia de las técnicas de estructuración de prompts en la exactitud funcional del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.	HE1. Las técnicas de estructuración de prompts influyen significativa y positivamente en la exactitud del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.		
PE2. ¿Cómo influyen las técnicas de estructuración de prompts en la mantenibilidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o?	OE2. Determinar la influencia de las técnicas de estructuración de prompts en la mantenibilidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.	HE2. Las técnicas de estructuración de prompts influyen significativ y positivamente en la mantenibilidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.		
PE3. ¿Cómo influyen las técnicas de estructuración de prompts en la complejidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o?	OE3. Determinar la influencia de las técnicas de estructuración de prompts en la complejidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.	HE3. Las técnicas de estructuración de prompts mejoran significativamente en la complejidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.		

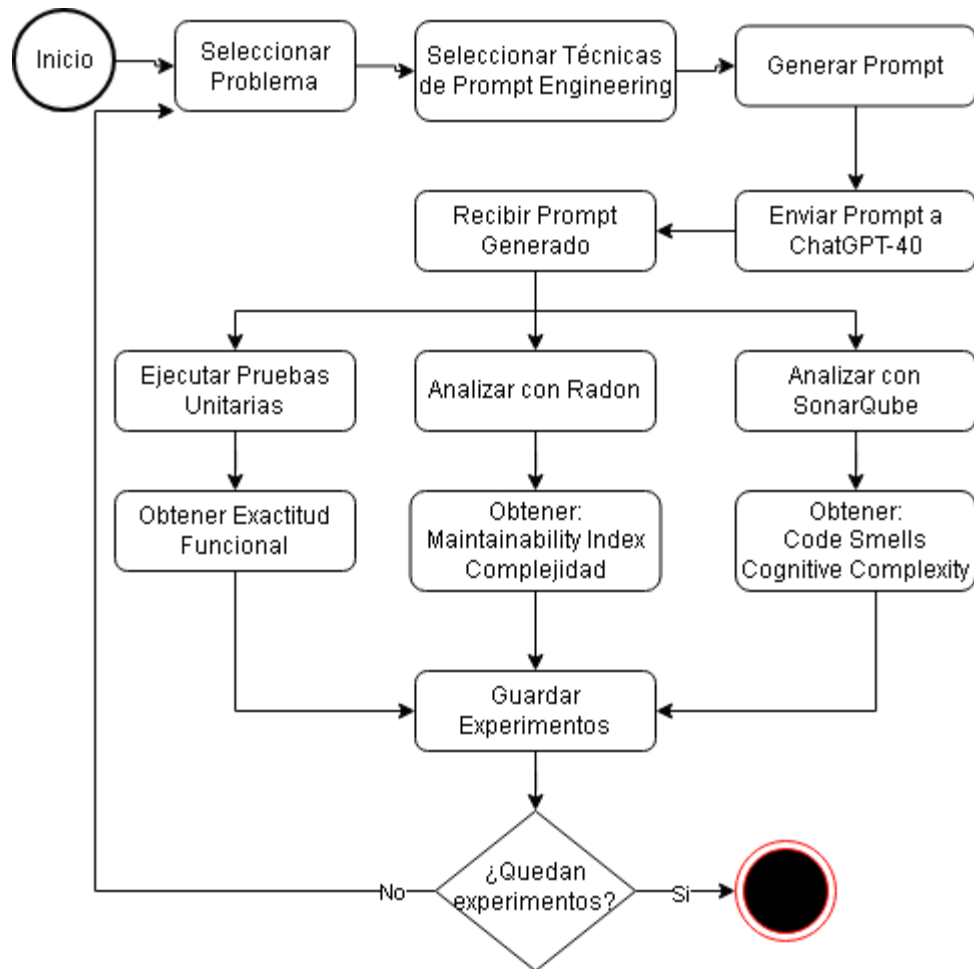
Anexo B. Tabla de Operacionalización de Variables

Variable	Parametros de Control	Tratamientos	Definición Operacional	Técnica	Instrumento	Escala
Variable Independiente: Técnicas de estructuración de prompts	• LLM: ChatGPT-4° • Temperatura: 0.0 • Lenguaje: Python • Banco de problemas: 30 ejercicios de LeetCode	Zero-Shot	Prompt sin ejemplos	No aplica	No aplica	Nominal
		Few-Shot	Prompt con ejemplos			
		Chain-of-Thought	Prompt con razonamiento paso a paso			
		Role Prompting	Prompt basado en roles			
Variable	Dimensiones	Indicadores	Definición Operacional	Técnica	Instrumento	Escala
Variable Dependiente: Calidad del código generado	Exactitud funcional	% pruebas superadas	Relación entre pruebas aprobadas y ejecutadas	Análisis documental	Ficha de registro experimental	Razón
	Mantenibilidad	Maintainability Index	Índice de mantenibilidad calculado por análisis estático			
		Technical Debt Ratio	Porcentaje estimado de deuda técnica			
	Complejidad	Complejidad ciclomática	Número de caminos independientes de ejecución			

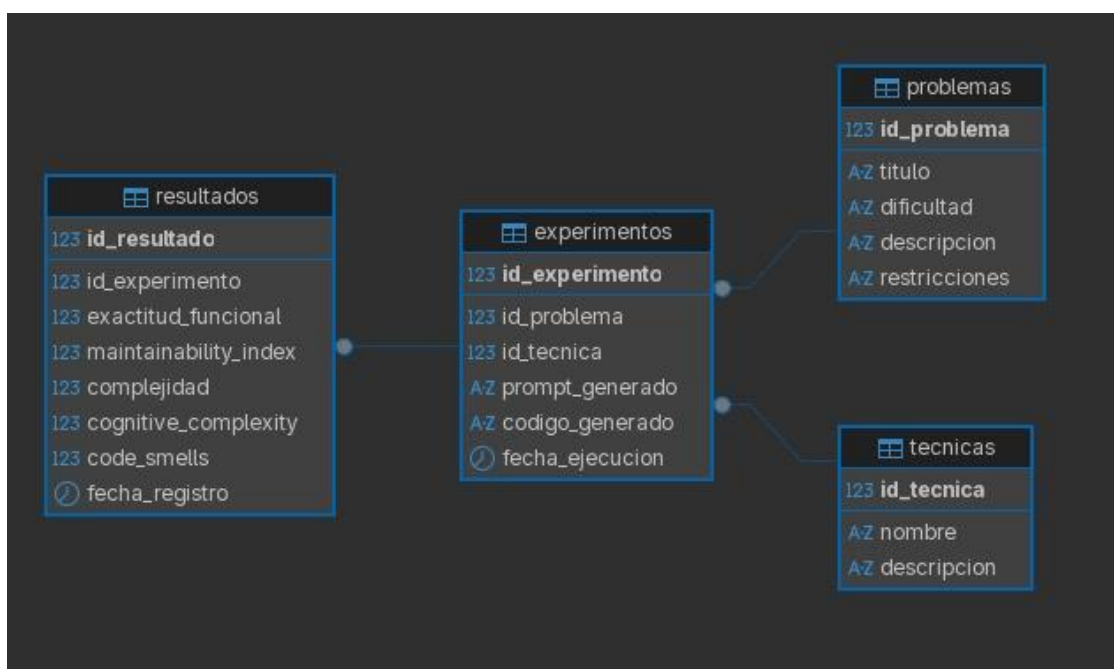
Anexo C. Diagrama de Componentes de la Plataforma de Experimentación



Anexo D. Diagrama de Flujo Experimental(Diagrama de Actividades)



Anexo E. Diagrama de Entidad Relación de la Base de Datos experimental



Anexo F. Ficha de Registro Experimental

FICHA DE REGISTRO EXPERIMENTAL

PARA LA EVALUACIÓN DE LA CALIDAD DEL CÓDIGO GENERADO

Proyecto: *Influencia de las técnicas de estructuración de prompts en la calidad del código generado por el modelo de lenguaje de gran escala ChatGPT-4o.*

Modo de registro de datos: La presente ficha constituye un instrumento de registro experimental automatizado. Los datos consignados son obtenidos directamente por la plataforma de evaluación desarrollada para la investigación, mediante la ejecución de pruebas funcionales automatizadas y herramientas de análisis estático de código. En consecuencia, el registro de la información no depende de la percepción o juicio de un evaluador humano, garantizando objetividad, consistencia y reproducibilidad en la recolección de datos.

Código de registro (ID_RESULTADO):

Fecha de evaluación:

I. DATOS DEL EXPERIMENTO

ID del experimento (ID_EXPERIMENTO):

Problema evaluado (ID_PROBLEMA):

Título del problema:

Fecha y Hora de ejecución:

Tiempo de Respuesta del modelo:

Nivel de dificultad:

- ☐ Fácil
- ☐ Medio
- ☐ Difícil

Lenguaje de programación:

Python

Modelo utilizado:

ChatGPT-4o

Técnica de estructuración de prompt aplicada:

- () Zero-Shot Prompting
- () Few-Shot Prompting
- () Chain-of-Thought Prompting
- () Role Prompting

II. REGISTRO DE MÉTRICAS DE CALIDAD DEL CÓDIGO

DIMENSIÓN 1: EXACTITUD FUNCIONAL

Nº total de pruebas	Nº de pruebas superadas	Porcentaje superadas (%)

Rango esperado: 0% – 100%

Fuente: Pruebas Unitarias

DIMENSIÓN 2: MANTENIBILIDAD

Indicador 1: Maintainability Index (MI)

Valor obtenido:

Rango: 0 puntos – 100 puntos

Fuente: SonarQube

Rango	Interpretación
85 – 100 puntos	Muy alta mantenibilidad
65 – 84 puntos	Buena mantenibilidad
40 – 64 puntos	Mantenibilidad moderada
20 – 39 puntos	Mantenibilidad baja
0 – 19 puntos	Muy baja mantenibilidad

Indicador 2: Technical Debt Ratio (TDR)

Valor obtenido (%):

Rango: 0% - 100%

Fuente: SonarQube

Rango	Interpretación
0% – 5%	Muy baja deuda técnica
5.1% – 10%	Baja deuda técnica
10.1% – 20%	Moderada deuda técnica
> 20%	Alta deuda técnica

DIMENSIÓN 3: COMPLEJIDAD

Indicador: Complejidad Ciclomática

Valor obtenido:

Rango: 1 o superior (valor entero)

Fuente: Radon

Rango	Interpretación
1 – 5	Muy simple
6 – 10	Simple
11 – 20	Moderadamente compleja
21 – 50	Compleja
> 50	Muy compleja

III. RESULTADOS CONSOLIDADOS

Indicador	Valor	Unidad
Porcentaje de pruebas superadas		%
Maintainability Index (MI)		puntos
Technical Debt Ratio (TDR)		%
Complejidad Ciclomática		valor entero

IV. OBSERVACIONES

Anexo G. Ficha de validación de Experto 1



UNIVERSIDAD CONTINENTAL
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS DE
INFORMATICA

FICHA DE VALIDACION DE INSTRUMENTO DE INVESTIGACIÓN

I. DATOS GENERALES

- 1.1 Título de la Investigación:** Técnicas de Estructuración de Prompts en la Calidad de Código Generado por el Modelo de Lenguaje de Gran Escala ChatGPT-4o
- 1.2 Apellidos y nombres del experto:** Sota Orellana Luis Alberto
- 1.3 Especialidad del experto:**
- 1.4 Autor del instrumento:** Harold Huanca Ccasa, Jose Leonardo Cruz Gonzales, Wilder Ronald Ccasani Quispe
- 1.5 Nombre del instrumento:** Ficha de Registro Experimental para la Evaluación de la Calidad de Código Generado

INDICADORES	CRITERIOS CUALITATIVOS/ CUANTITATIVOS	Deficiente 0-20%	Regular 21-40%	Bueno 41-60%	Muy Bueno 61-80%	Excelente 81-100%
1. CLARIDAD	Está formulado con lenguaje apropiado.					90%
2. OBJETIVIDAD	Está expresado en conductas observables.				80%	
3. ACTUALIDAD	Adecuado al alcance de ciencia y tecnología.					95%
4. ORGANIZACIÓN	Existe una organización lógica.					90%
5. SUFICIENCIA	Comprende los aspectos de cantidad y calidad.					85%
6. INTENCIONALIDAD	Adecuado para valorar aspectos del estudio.					90%
7. CONSISTENCIA	Basados en aspectos Teóricos-Científicos y del tema de estudio.					90%
8. COHERENCIA	Entre indicadores, dimensiones, variables o categorías.					95%
9. METODOLOGÍA	La estrategia responde al propósito del estudio.					85%
10. CONVENIENCIA	Genera nuevas pautas en la investigación y construcción de teorías.					90%
TOTAL						

II. DATOS DE VALORACIÓN

2.1 VALORACION CUANTITATIVA (Total x 0.10): $890 \times 0.10 = 89.0$

2.2 VALORACIÓN CUALITATIVA: EXCELENTE

2.3 OPINIÓN DE APLICABILIDAD: El instrumento es aplicable

Lugar y fecha: Cusco, 18 de junio del 2026


Dr. Luis Alberto Sota Orellana
INGENIERO DE SISTEMAS
DNI: 723981520

Anexo H. Ficha de validación de Experto 2



UNIVERSIDAD CONTINENTAL
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS
DE INFORMÁTICA

FICHA DE VALIDACION DE INSTRUMENTO DE INVESTIGACIÓN

I. DATOS GENERALES

1.1 **Título de la Investigación:** Técnicas de Estructuración de Prompts en la Calidad de Código
Generado por el Modelo de Lenguaje de Gran Escala ChatGPT-4o

1.2 **Apellidos y nombres del experto:** Gutiérrez Huaman Nestor

1.3 **Especialidad del experto:** Mg. Gestión Empresarial

1.4 **Autor del instrumento:**

Harold Huanca Ccasa, Jose Leonardo Cruz Gonzales, Wilder Ronald Ccasani Quispe

1.5 **Nombre del instrumento:** Ficha de Registro Experimental para la Evaluación de la
Calidad del Código Generado

INDICADORES	CRITERIOS CUALITATIVOS/ CUANTITATIVOS	Deficiente 0-20%	Regular 21-40%	Bueno 41-60%	Muy Bueno 61-80%	Excelente 81-100%	
1. CLARIDAD	Está formulado con lenguaje apropiado.					X	90%
2. OBJETIVIDAD	Está expresado en conductas observables.					X	100%
3. ACTUALIDAD	Adecuado al alcance de ciencia y tecnología.					X	95%
4. ORGANIZACIÓN	Existe una organización lógica.					X	90%
5. SUFICIENCIA	Comprende los aspectos de cantidad y calidad.					X	85%
6. INTENCIONALIDAD	Adecuado para valorar aspectos del estudio.					X	95%
7. CONSISTENCIA	Basados en aspectos Teóricos-Científicos y del tema de estudio.					X	90%
8. COHERENCIA	Entre indicadores, dimensiones, variables o categorías.					X	90%
9. METODOLOGÍA	La estrategia responde al propósito del estudio.					X	100%
10. CONVENIENCIA	Genera nuevas pautas en la investigación y construcción de teorías.					X	85%
TOTAL							920

II. DATOS DE VALORACIÓN

2.1 **VALORACION CUANTITATIVA (Total x 0.10):** 92

2.2 **VALORACIÓN CUALITATIVA:** Excelente

2.3 **OPINIÓN DE APLICABILIDAD:** Es Aplicable

Lugar y fecha: Cusco, 18 de Junio del 2026


Dr. Nestor Gutierrez Huaman
DNI: 60259662

Anexo I. Ficha de validación de Experto 3



UNIVERSIDAD CONTINENTAL
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS
DE INFORMÁTICA

FICHA DE VALIDACION DE INSTRUMENTO DE INVESTIGACIÓN

I. DATOS GENERALES

- 1.1 Título de la Investigación:** Técnicas de Estructuración de Prompts en la Calidad de Código Generado por el Modelo de Lenguaje de Gran Escala ChatGPT-4o
- 1.1 Apellidos y nombres del experto:** Jorge Mendoza Quispe
- 1.2 Especialidad del experto:** Ingeniero de Datos
- 1.3 Autor del instrumento:** Harold Huanca Ccasa, Jose Leonardo Cruz Gonzales, Wilder Ronald Ccasani Quispe
- 1.2 Nombre del instrumento:** Ficha de Registro Experimental para la Evaluación de la Calidad de Código Generado

INDICADORES	CRITERIOS CUALITATIVOS/ CUANTITATIVOS	Deficiente 0-20%	Regular 21-40%	Bueno 41-60%	Muy Bueno 61-80%	Excelente 81-100%
1. CLARIDAD	Está formulado con lenguaje apropiado.					95%
2. OBJETIVIDAD	Está expresado en conductas observables.					90%
3. ACTUALIDAD	Adecuado al alcance de ciencia y tecnología.					90%
4. ORGANIZACIÓN	Existe una organización lógica.					90%
5. SUFICIENCIA	Comprende los aspectos de cantidad y calidad.					85%
6. INTENCIONALIDAD	Adecuado para valorar aspectos del estudio.					90%
7. CONSISTENCIA	Basados en aspectos Teóricos Científicos y del tema de estudio.					90%
8. COHERENCIA	Entre indicadores, dimensiones, variables o categorías.					90%
9. METODOLOGÍA	La estrategia responde al propósito del estudio.					90%
10. CONVENIENCIA	Genera nuevas pautas en la investigación y construcción de teorías.				80%	
TOTAL		890				

II. DATOS DE VALORACIÓN

2.1 VALORACION CUANTITATIVA (Total x 0.10): 89

2.2 VALORACIÓN CUALITATIVA: Excelente.

2.3 OPINIÓN DE APLICABILIDAD: Apta para aplicarse.

Lugar y fecha: Cusco, 22 de Junio del 2026


 JORGE MENDOZA QUISPE
 GERENTE GENERAL
 CODE INVERSIONES S.A.C.
 RUC: 20450536229
 Jorge Mendoza Quispe